

BrandBridge Build Manual

The single document five engineers build from: architecture, database tables, API contracts, and a step-by-step construction order for both directions of discovery — brand → creator and creator → brand.

REPOSITORY	REPO STATE READ	TEAM	DELIVERY WINDOW	CAPACITY
Harsh-Gupta19/BrandBridge_AI	200 files · 10 commits · Phase 1	Harshit, Mayank, Manjusha, Lakshmi, Mohit	7 weekends · 25 Sep → 8 Nov 2026	105 person-days

How to use this manual

Five PDFs define *what* BrandBridge is and *why* it is academically defensible. This manual defines *how* it gets built, in what order, into which tables, behind which endpoints.

Source precedence

When two documents disagree, resolve in this order. Everything below was reconciled against the repository as it actually stands today, not as the documents assume it stands.

RANK	SOURCE	AUTHORITATIVE FOR
1	Consolidated Project Plan V3 RESEARCH	Research question, sub-problems 1–14, scope boundaries, MVP vs. stretch, evaluation design
2	Technical Architecture & Implementation Guide	Monorepo layout, schema intent, API surface, ADRs, social integration path
3	Creator Profile & Data Strategy	Creator evidence model, multi-vertical representation, dataset scale targets
4	Business & Academic Structure	Problem framing, personas, demo narrative
5	UI Wireframes PROVISIONAL	Client-side baseline only — explicitly not frozen
6	This manual	Build order, exact DDL, endpoint contracts, file paths, ownership, definition of done

THE ONE RULE THAT KEEPS THIS PROJECT SHIPPABLE

Every AI component in BrandBridge is **replaceable by a deterministic fallback**. The LLM parser falls back to a form. Social APIs fall back to manual profile entry. Semantic retrieval falls back to SQL category filters. If you cannot demo the flow with the AI switched off, the flow is not done — because on demo day something will be rate-limited.

What this manual adds that the existing documents do not have

- **A repo-anchored gap list.** Part 1 compares the target design against the 200 files that exist today, so nobody re-designs what is already built or assumes something exists that does not.
- **Executable table structures.** Part 5 gives real `CREATE TABLE` statements and a numbered Alembic migration sequence, including the breaking change your current `users.role` column requires.
- **Both directions specified symmetrically.** The existing guide specifies brand → creator in detail and leaves creator → brand as one endpoint. Part 7 specifies both as first-class flows with different queries, different gates and different caches.
- **Step-level construction order.** Part 7 walks each flow from the screen down to the SQL, with the file to open at every step.
- **A UI review with the missing screens named.** Part 9.

Where the repository actually is

An honest scoreboard, read directly from the clone. The foundation is genuinely good — the monorepo, Docker topology, CI and module boundaries are all in place and correct. What is missing is every line of domain behaviour.

5	37	6	0	0	200
ORM MODELS	TABLES NEEDED	PLACEHOLDER ROUTES	VECTOR COLUMNS	REPOSITORIES	CREATORS COLLECTED

1.1 Component-by-component status

AREA	IN THE REPO TODAY	STATUS	WHAT IS REQUIRED
Monorepo & folder boundaries	Full tree: <code>frontend/ backend/ notebooks/ data/ docs/ infra/ scripts/</code> , AI and ML sub-packages stubbed	DONE	Keep as is. Do not restructure.
Docker Compose	<code>pgvector/pgvector:pg16</code> , backend, frontend, healthchecks, named volume	DONE	Add an <code>alembic upgrade head</code> entrypoint step.
CI	<code>.github/workflows/ci.yml</code> , PR + issue templates	DONE	Add a Docker smoke test in week 6.
FastAPI shell	<code>main.py</code> , CORS, settings via <code>pydantic-settings</code> , <code>/health</code> + <code>/api/v1/health</code>	DONE	Add error-shape middleware and <code>correlation_id</code> .
Password hashing	<code>core/security.py</code> – PBKDF2-SHA256, 210k iterations, constant-time compare	DONE	Correct and dependency-free. Keep it; do not add <code>passlib</code> . Add JWT issue/verify beside it.
Authentication	Nothing. <code>auth.py</code> returns a placeholder object.	MISSING	Register, login, refresh, <code>get_current_user</code> , role guards. Blocks everything.
Dual-mode identity	<code>users.role</code> is a single enum column	BLOCKING	One account must carry Brand <i>and</i> Creator. Needs a <code>user_roles</code> table. See §5.4.
Creator profile	6 columns: <code>display_name</code> , bio, location, categories, <code>audience_summary</code>	PARTIAL	Needs ~14 more columns plus <code>creator_preferences</code> , <code>rate_card_items</code> , <code>creator_availability</code> .
Campaign	<code>title</code> , brief, <code>budget_amount</code> , <code>target_*</code> , platforms, formats	PARTIAL	Split into <code>campaigns</code> (raw + lifecycle) and <code>campaign_requirements</code> (confirmed structure). Raw and parsed must both survive.
Proposal	<code>campaign_id</code> , <code>creator_profile_id</code> , status, message, offer	PARTIAL	No <code>initiated_by</code> column – this alone blocks the two-way product . Also needs COUNTER_OFFER and versions.
Repositories layer	<code>app/repositories/</code> exists and is empty	MISSING	Every table gets one module. Services never write SQL.
Services layer	<code>services/health.py</code> only	MISSING	7 domain services. See §3.2.
pgvector	Extension created in migration <code>0001</code> ; no <code>VECTOR</code> column anywhere	MISSING	<code>creator_representations</code> , <code>creator_content_clusters</code> , <code>document_chunks</code> .
ML modules	<code>ml/features/planned_features.py</code> – naming only	MISSING	Feature builder, 4-model ladder, evaluation harness.
AI modules	<code>ai/agents/planned_workflow.py</code> , <code>ai/tools/planned_tools.py</code> – naming only	MISSING	LLM adapter, embedder, RAG, 9 tools, LangGraph.
Data collectors	YouTube, Instagram (business + creator), Bluesky collectors; 200 real creator profiles already collected with 1,778 videos and 4,434 posts	AHEAD	The most valuable asset in the repo and further along than any document assumes. They must become <code>integrations/</code> adapters writing to tables rather than files. See §1.3–1.4.
Transformed creator contract	<code>data/samples/transformed_creator.sample.json</code> – excellent: provenance, data-quality scores, sample windows	KEEP	Promote this to the canonical normalised contract. Part 5 maps it field-for-field onto tables.
Frontend	11 route pages, all <code>PagePlaceholder</code> ; <code>apiClient</code> has GET only; no auth	PARTIAL	Auth context, active-role switch, typed client with POST/PATCH, forms, 11 new screens.
Documents & RAG	Nothing	MISSING	<code>documents</code> , <code>document_chunks</code> , upload endpoint, retriever.
Match persistence	Nothing	MISSING	<code>match_scores</code> with component columns. Explainability depends entirely on this.

1.2 Three decisions to close in weekend 1

These are not research questions. They are build-blocking and each has a recommended answer you can adopt in ten minutes.

D1 · Identity model

Question: single role per user, or one account with two activatable profiles?

Recommendation: one account, two profiles. The wireframes, the demo script and the reciprocal research story all require a user to switch modes. Cost: one extra table, one migration, one `X-Active-Role` header.

D2 · Third platform

Question: Bluesky is in the repo but in none of the documents. Keep it?

Recommendation: keep it as a *third adapter*, not a third product surface. It has an open, key-free API, which makes it the most reliable demo-day data source you have. Declare it in an ADR and normalise it through the same contract.

D3 · The Wall SETTLED

Decided: the Wall ships. It is a peer community where brands and creators post ideas, open calls and results, follow each other and reply in threads — the surface that attracts people to the platform before any campaign exists.

Two constraints keep it safe: wall content never enters the ranking pipeline, and a thread converts into an ordinary proposal rather than becoming a second negotiation channel. Cost: four tables, eight endpoints, two screens. See §5.12 and F13.

1.3 You already have the dataset — this changes the plan

The research documents plan for 150–200 creator profiles as a minimum and 300–400 as a goal, treating acquisition as a risk. Reading `data/raw/` shows you are already at the minimum, with real content attached.

SOURCE	CREATORS	CONTENT ITEMS	COLLECTED	ALREADY NORMALISED?
YouTube	100	1,778 videos	13 & 17 Sep 2026	Yes — each record carries <code>derived_features</code> , <code>recent_video_metrics</code> and a <code>brandbridge_fields</code> block
Bluesky	100	4,434 posts	14 Sep 2026	Yes — same shape, plus <code>recent_post_metrics</code>
Instagram	—	—	collectors written, not yet run at scale	n/a
Total	200	6,212	—	—

WHAT THIS MEANS FOR THE SCHEDULE

Dataset acquisition is not the project risk the documents assume, so weekend 2 should **seed from these 200 real profiles** rather than generating synthetic ones. Spend the saved time on the two things that are still genuinely scarce: **labelled pairs** and the **30–50 consented creators** who can supply real rates, exclusions and audience data. Those cannot be collected in an afternoon and they are what make the evaluation defensible.

The gap in this data is commercial, not observational: you have follower counts, engagement and content text, and you have no rate cards, no accepted or excluded categories and no availability. Those are the creator-side fields S_c is computed from — so generate them as clearly-labelled `SYNTHETIC` attributes for the research set, and use the consented gold set to check whether the synthetic distribution was plausible.

1.4 Two repository problems to fix in weekend 1

PROBLEM 1 — THE WORKING TREE IS 509 MB

`data/raw/` holds 509 MB across fifteen JSON files, and individual files reach 61 MB. That is slow to clone on a student connection, and single files above 50 MB draw a GitHub warning while anything above 100 MB is rejected outright — you are one collection run away from a push that fails.

The cause is threefold duplication rather than volume. Each creator record embeds its own `recent_videos` (or `recent_posts`) and a full `raw` provider payload; those same items are then stored again in the separate `*_videos` / `*_posts` file; and the combined `*_dataset` file stores both a third time. On top of that, every run is committed twice — once timestamped and once as `.latest`.

Fix, roughly one hour: keep one canonical `*_dataset.latest.json` per platform; drop the `raw` key from each creator (it is a provider payload you can re-fetch, not evidence you need); reference content by id from the creator instead of embedding it; and add `data/raw/**/*_2026*.json` to `.gitignore` so timestamped runs stay local. That removes well over 90% of the weight with no loss of anything the pipeline reads. Then rewrite history with `git filter-repo` so the old blobs actually leave the pack — deleting the files in a new commit does not shrink a clone.

PROBLEM 2 – THE COLLECTORS WRITE FILES, NOT ROWS

Keep the collection logic; it is good, and the `brandbridge_fields` blocks show the normalisation thinking is already there. Change only where it writes. Each collector becomes `backend/app/integrations/<platform>/adapter.py` exposing one function that returns the **normalised social contract** of §5.7 — never a platform-shaped dict. Then `scripts/seed_data.py`, `scripts/sync_social.py` and the live OAuth endpoint all call the same adapter, and the ML layer never learns that YouTube counts subscribers while Bluesky counts followers.

Also move `instagram_data.json` out of the repository root into `data/raw/instagram/`, and confirm no access token was committed inside it.

High-level architecture

One React client in two modes, one FastAPI deployable, one PostgreSQL. Every external dependency — LLM, YouTube, Meta, Bluesky — sits behind an adapter and has a defined behaviour when it is unavailable.

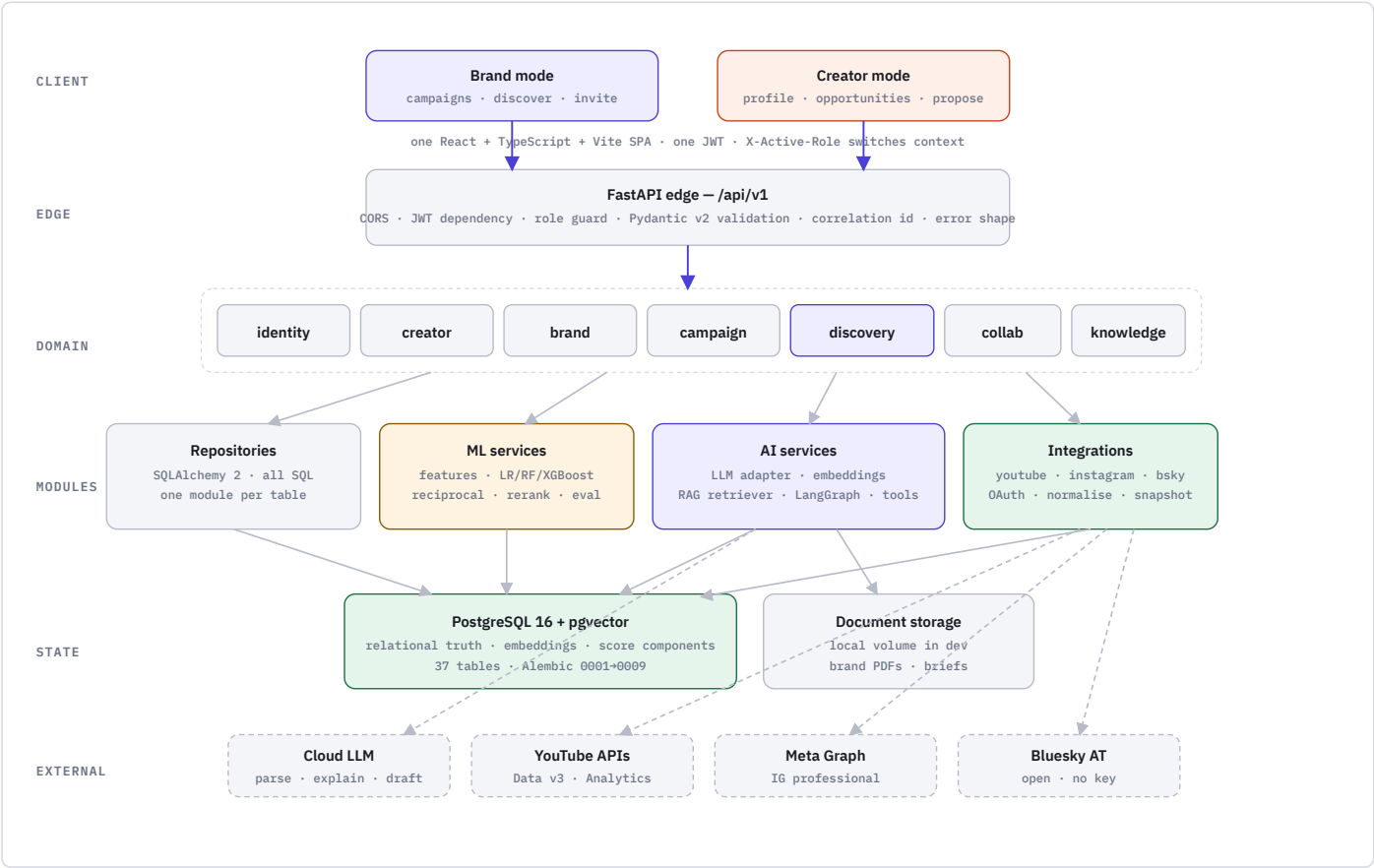


Figure 1 — Container view. Dashed boundaries are external and optional: every one of them has a defined offline behaviour. The discovery service is highlighted because it is the only service that touches ML, AI and repositories at once.

2.1 Seven runtime principles

#	PRINCIPLE	ENFORCED BY
1	Hard constraints are evaluated before any scoring, and never inside a model	<code>services/eligibility.py</code> runs first in the pipeline and writes an exclusion reason row
2	Every score that reaches a screen was persisted first, with its component parts	<code>match_scores</code> is written inside the same transaction that returns the top-K
3	The LLM receives stored facts; it never produces a number	Explanation input is a typed <code>MatchEvidence</code> object, not the creator row
4	RAG answers questions about documents; SQL answers questions about the marketplace	Retriever is only reachable from <code>knowledge_service</code>
5	No commercial action is taken by software	Proposal, invitation, counter and accept all require an authenticated POST from a human
6	Provider data is a dated snapshot, never mutable truth	<code>social_metric_snapshots</code> is append-only; the UI shows <code>captured_at</code>
7	External failure degrades one feature, never the demo	Each adapter returns <code>(data, freshness, degraded_reason)</code> ; callers must handle <code>degraded</code>

2.2 Deployment topology

Unchanged from the repo, which is already right. Three containers locally; two deployable units plus a managed database in the cloud. No Kubernetes, no message broker, no separate vector store — those are explicitly out of scope and an ADR already records why.

UNIT	LOCAL	CLOUD	SCALES BECAUSE
Frontend	Vite dev server :5173	Static build on any CDN host	It is a static bundle
Backend	Uvicorn :8000 in container	One container instance	Stateless apart from the DB; long AI jobs go to BackgroundTasks
Database	pgvector/pgvector:pg16 :5432	Managed Postgres with the vector extension	Single writer is plenty at 400 creators
Documents	Mounted volume	Object storage bucket	Only documents.storage_uri changes

Low-level architecture

The layer contract. Five people writing into one backend need one rule about who may call what — this is that rule, and code review enforces it.

3.1 The layer contract

LAYER	PATH	MAY DO	MUST NEVER DO
Router	api/v1/endpoints/*.py	Declare path, method, response model, dependencies. Call exactly one service function. Map domain errors to HTTP.	Touch a session, build a query, call an ML or AI module, contain an <code>if</code> about business rules
Dependency	api/deps.py	Decode JWT, load user, resolve active role, assert profile ownership, yield <code>Session</code>	Contain domain logic
Schema	schemas/*.py	Pydantic v2 request/response models, field validation, examples	Import a SQLAlchemy model
Service	services/*.py	Orchestrate: call repositories, ML, AI, integrations. Own the transaction boundary. Raise domain errors.	Write raw SQL, import <code>fastapi</code> , know about HTTP status codes
Repository	repositories/*.py	Every SELECT/INSERT/UPDATE. Return ORM objects or typed rows.	Commit, call another repository, call a service
Model	models/*.py	Table definitions, relationships, constraints	Contain methods with business meaning
ML	ml/{features,inference,training,evaluation}	Take a plain feature dict or DataFrame, return numbers plus a <code>feature_version</code>	Query the database, import a service
AI	ai/{llm,embeddings,rag,agents,tools,prompts}	Provider calls, embedding, retrieval, graph orchestration. Return typed objects.	Own a transaction, decide eligibility, invent a metric
Integration	integrations/{youtube,instagram,bsky}	OAuth exchange, provider HTTP, normalise to the \$5.7 contract	Write to the database directly, leak a provider-shaped dict upward

ONE-LINE REVIEW TEST

If a file under `api/` contains the word `select(`, or a file under `ml/` contains the word `Session`, the pull request is rejected. That single check keeps five parallel contributors from fusing the layers together in week 3.

3.2 Request path and module decomposition

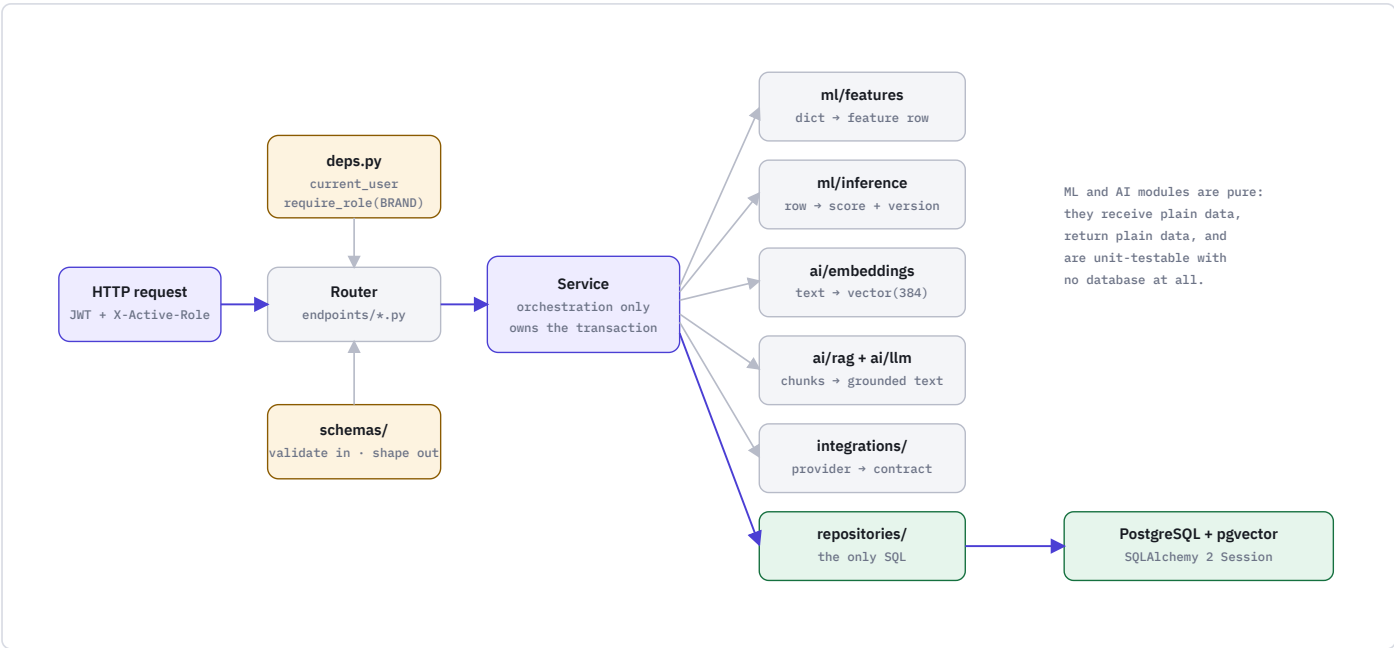


Figure 2 — The service is the only layer allowed to know about more than one capability module. This is what makes `calculate_ml_match` and `calculate_semantic_similarity` testable in isolation, which the evaluation work in Part 10 depends on.

3.3 Target backend tree

Existing paths are unmarked; **NEW** marks a file you create. The tree itself does not change — only its contents.

BACKEND/APP/ – TARGET STATE AT WEEK 7

```
app/
├── main.py                + error handler, correlation-id middleware
├── core/
│   ├── config.py         + jwt_expiry, embedding_model, oauth_token_key
│   ├── security.py       PBKDF2 kept · + create_access_token / decode_token
│   ├── logging.py
│   └── errors.py         NEW DomainError hierarchy + HTTP mapping
├── api/
│   ├── deps.py           NEW get_db · get_current_user · require_role · active_profile
│   └── v1/
│       ├── router.py     + documents, integrations, collaborations, ai
│       └── endpoints/
│           ├── auth.py   REPLACE register · login · refresh · me
│           ├── creators.py REPLACE profile · preferences · rate-card · representation
│           ├── brands.py  REPLACE profile · documents
│           ├── campaigns.py REPLACE CRUD · parse-brief · confirm · publish
│           ├── recommendations.py REPLACE generate · read · creator-side · match detail
│           ├── proposals.py REPLACE propose · invite · counter · accept · reject
│           ├── collaborations.py NEW list · detail · status · messages · agreement
│           ├── documents.py NEW upload · list · knowledge query
│           ├── integrations.py NEW oauth connect/callback/sync/disconnect
│           └── ai.py      NEW proposal draft · run status
├── schemas/              NEW one module per domain noun
├── services/
│   ├── identity.py creator.py brand.py campaign.py NEW
│   ├── eligibility.py NEW deterministic hard filters
│   ├── discovery.py NEW the reciprocal pipeline, both directions
│   ├── collaboration.py NEW proposal state machine
│   └── knowledge.py NEW documents + RAG
├── repositories/        NEW one module per table group
├── models/              + 23 tables (see Part 5)
├── ml/
│   ├── features/builder.py NEW the single feature contract
│   ├── inference/ranker.py NEW load artifact, score, return feature_version
│   ├── inference/reciprocal.py NEW brand-side · creator-side · combination
│   ├── inference/rerank.py NEW MMR diversity with relevance floor
│   ├── training/train_ranker.py NEW
│   └── evaluation/metrics.py NEW P@K · R@K · NDCG@K · MAP · exposure · Gini
├── ai/
│   ├── llm/provider.py NEW one interface, one cloud implementation
│   ├── llm/structured.py NEW schema-constrained extraction + repair
│   ├── embeddings/encoder.py NEW sentence-transformers, cached singleton
│   ├── rag/{ingest,retriever}.py NEW
│   ├── tools/*.py NEW the 9 tools of §7.12
│   ├── agents/match_graph.py NEW LangGraph
│   └── prompts/*.md NEW versioned prompt text
├── integrations/
│   ├── base.py NEW SocialAdapter protocol + normalised contract
│   ├── youtube/adapters.py REFACTOR from youtubedatacollector.py
│   ├── instagram/adapters.py REFACTOR from instagramcollector*.py
│   └── bsky/adapters.py REFACTOR from bskydatacollector.py
```

Technology stack and why each piece exists

Most of this is already chosen and already installed. The table below is the complete list with the justification you will be asked for in the viva, plus the short list of additions the current `requirements.txt` and `package.json` are missing.

4.1 The stack in full

LAYER	CHOICE	PURPOSE IN BRANDBRIDGE	WHY NOT THE ALTERNATIVE	REPO
Client	React 18 + TypeScript	Two-mode SPA; typed contract against the OpenAPI schema	Types catch the brand/creator payload divergence at compile time	IN
	Vite 5	Dev server, build, env injection	Next.js SSR buys nothing here; there is no SEO surface	IN
	React Router 6	Role-aware routing and guards	—	IN
	TanStack Query 5	Server cache; recommendation lists are expensive and must not refetch on every render	Hand-rolled <code>useEffect</code> fetching cannot express the stale/invalidate rules this product needs	IN
	Vitest + Testing Library	Component smoke tests	—	IN
API	Python 3.12 + FastAPI	The ML and LLM libraries are Python; keeping the API in Python removes a serialisation hop between the ranker and the endpoint	A Node API would force an RPC boundary to the model	IN
	Pydantic v2	Request/response validation <i>and</i> the schema the LLM extraction is constrained to	One schema definition serves both the API contract and the parser contract	IN
	SQLAlchemy 2 + Alembic	ORM, typed <code>Mapped[]</code> columns, versioned migrations	Raw SQL across five contributors drifts; Alembic makes schema changes reviewable	IN
	psycopg 3	Postgres driver	—	IN
	PyJWT	Access + refresh tokens	Lighter than <code>python-jose</code> ; you only need HS256	ADD
	python-multipart	Document upload (FR-11)	FastAPI cannot accept a file upload without it	ADD
Data	PostgreSQL 16	Source of truth: identity, profiles, campaigns, rates, proposals, score components, labels	—	IN
	pgvector	Creator representation vectors, cluster vectors, document chunk vectors, ANN search	A separate vector DB adds an operational component and a consistency problem for 400 creators	EXT ONLY
	<code>pgvector</code> Python package	The SQLAlchemy <code>Vector</code> column type	Without it you cannot declare a vector column in the ORM	ADD
ML	pandas + numpy	Feature frames, evaluation tables	—	IN
	scikit-learn	Logistic Regression and Random Forest baselines; train/test splitting; metrics	The interpretable baselines are the control condition of the whole experiment	IN
	XGBoost	Primary structured ranker; optional <code>rank:pairwise</code> objective	Best fit for a few hundred labelled pairs of tabular features	IN
	joblib	Model artifact persistence into <code>ml/artifacts/</code> with a version string	Pickling by hand loses the version discipline NFR-03 requires	ADD
	SHAP OPTIONAL	Per-prediction feature contributions for the tree model, feeding explanations	Only if time remains; stored components already satisfy FR-08	ADD
AI	sentence-transformers	Local text encoder for creator bios, captions, campaign briefs, document chunks	Local, free, reproducible, no per-call cost for ~400×40 documents	IN
	LangChain	Provider abstraction, prompt templates, structured output parsing, text splitters, retriever interface	Only these pieces. It does not own business logic.	IN
	LangGraph	Stateful match workflow with an explicit human-approval interrupt	The checkpoint/interrupt model is exactly the human-in-the-loop requirement	IN
	A cloud LLM, provider-abstracted	Brief parsing, explanation narration, proposal drafting, grounded document Q&A	Four language tasks only; never scoring or filtering	KEY
	pypdf	Extracting text from uploaded brand guideline PDFs	—	ADD
Integrations	httplib	OAuth code exchange and provider REST calls, with timeouts	Async-capable and already a FastAPI transitive dependency	ADD
	tenacity	Bounded retry with backoff on provider 429/5xx	Hand-written retry loops hide quota exhaustion	ADD
	cryptography	Symmetric encryption of stored OAuth refresh tokens (NFR-01)	Tokens in plaintext columns fail the security review	ADD

LAYER	CHOICE	PURPOSE IN BRANDBRIDGE	WHY NOT THE ALTERNATIVE	REPO
Ops	Docker + Compose	One-command reproducible environment (NFR-02)	—	IN
	GitHub Actions	Lint, test, build on every PR	—	IN
	Ruff + pytest / ESLint + Prettier	Style and correctness gates	—	IN

4.2 Exactly what to install

BACKEND/REQUIREMENTS.TXT — APPEND THESE NINE LINES

```
PyJWT>=2.9.0,<3.0.0      # access + refresh tokens (HS256)
python-multipart>=0.0.9  # document upload  FR-11
pgvector>=0.3.0,<1.0.0   # SQLAlchemy Vector column type
httpx>=0.27.0,<1.0.0     # OAuth exchange + provider calls
tenacity>=9.0.0,<10.0.0  # bounded retry on 429/5xx
cryptography>=43.0.0     # encrypt stored refresh tokens  NFR-01
pypdf>=5.0.0,<6.0.0      # brand guideline PDF → text
joblib>=1.4.0,<2.0.0     # versioned model artifacts
langchain-text-splitters>=0.3.0 # recursive chunking for RAG

# optional, only if week 9 has room:
# shap>=0.46.0           # per-prediction feature contributions
```

FRONTEND/ — NPM INSTALL

```
npm i react-hook-form @hookform/resolvers zod zustand lucide-react
npm i -D tailwindcss postcss autoprefixer msw

# react-hook-form + zod → this product has 14 forms; the campaign brief form alone
#                        has 11 validated fields. Hand-rolled state will not hold.
# zustand                → auth token + activeRole. ~40 lines, no provider pyramid.
# tailwind               → the wireframes are dense, card-heavy and two-themed.
# lucide-react           → icon set for nav, status chips, platform badges.
# msw                   → mock the API so frontend work is never blocked on backend.
```

DELIBERATELY REJECTED — RECORD THESE AS ADR 011

Redis, Kafka, Kubernetes, Celery, Neo4j, Pinecone/Milvus/Qdrant, Feast/Hopsworks, a trained two-tower retrieval model, GNN embeddings, and MLflow. Each is a credible production choice and none is justified at 400 creators inside seven weekends. Use FastAPI [BackgroundTasks](#) where you would have reached for Celery, and a versioned CSV plus a markdown experiment log where you would have reached for MLflow. Being able to defend an omission is worth more marks than an unused dependency.

Database architecture and table structures

37 tables in nine groups. Every table here is reachable from a flow in Part 7 — nothing is speculative. The five that exist today are marked, including the one that has to change shape.

5.1 Storage principles

- **Postgres is the truth; pgvector is an index on it.** No fact lives only in a vector.
- **Raw and derived both survive.** `campaigns.raw_brief` is never overwritten by the parsed structure — the extraction experiment in Part 10 needs both.
- **Provider data is append-only and dated.** A follower count is `(value, captured_at, provenance)`, never a mutable column.
- **Scores are stored with their parts and their versions.** A row in `match_scores` carries `feature_version` and `model_version`, so a result from week 4 stays interpretable in week 7.
- **Real and synthetic are always distinguishable.** Every evidence-bearing row carries a `data_provenance` value; the research dataset can be filtered on it.
- **Commercial history is versioned, never edited.** A counter-offer creates a new `proposal_versions` row.

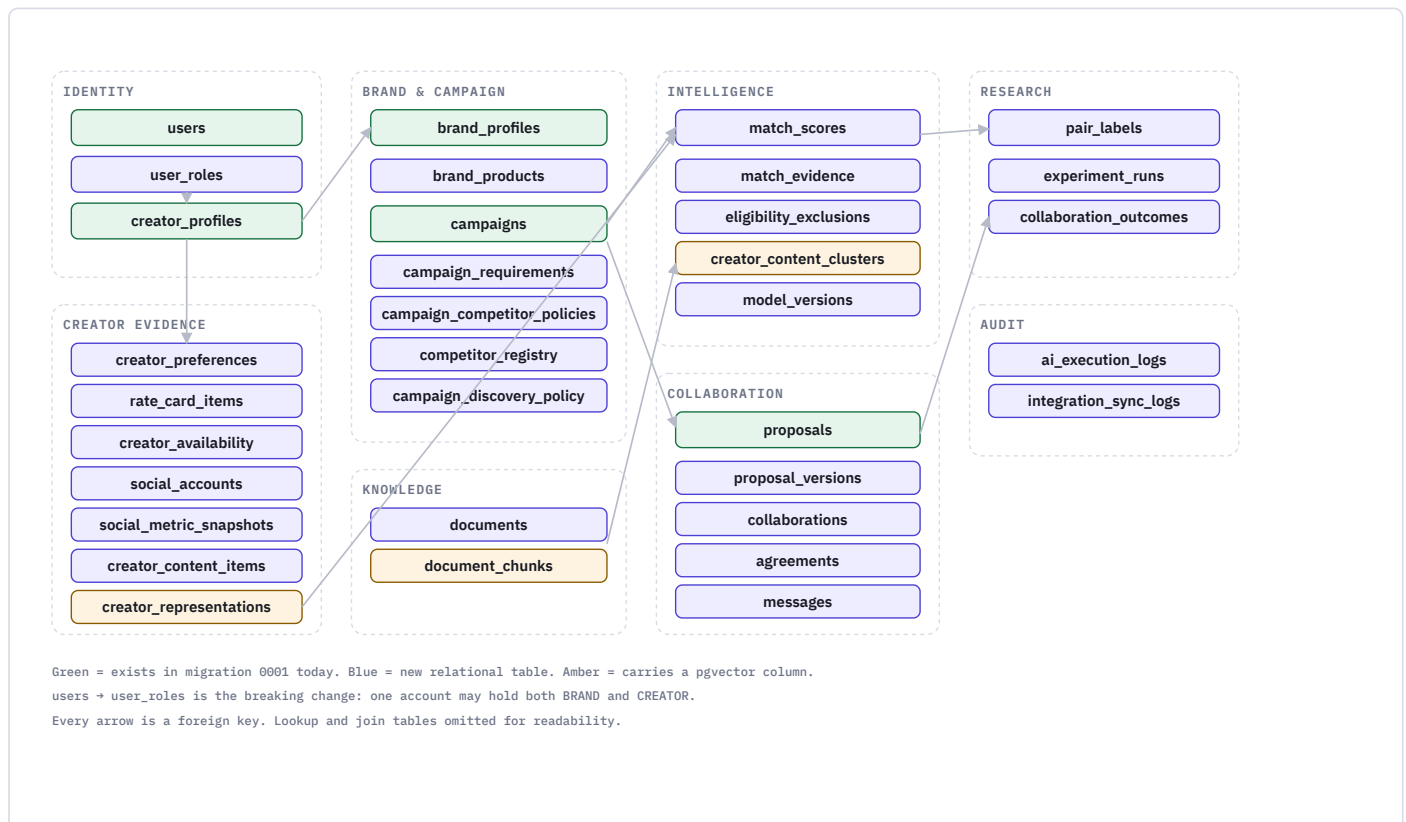


Figure 3 — Entity groups. Read left to right: identity produces evidence, evidence meets campaign requirements in the intelligence group, and the intelligence group produces the rows that collaboration acts on.

5.2 Migration sequence

Eight revisions, each independently reviewable and each tied to the weekend that needs it. Never edit `0001`; it is already applied.

REVISION	CONTENTS	ENABLES	WEEKEND
0001_initial_foundation	users, creator_profiles, brand_profiles, campaigns, proposals, CREATE EXTENSION vector	—	APPLIED
0002_dual_role_identity	user_roles ; backfill from users.role ; drop users.role ; add users.status	F1 — one account, two modes	W1
0003_creator_commercial	Extend creator_profiles ; add creator_preferences , rate_card_items , creator_availability	F2, F3 — profile + rate card	W1
0004_social_evidence	social_accounts , social_metric_snapshots , creator_content_items , integration_sync_logs	F4 — connect and sync	W2
0005_campaign_requirements	Extend campaigns ; add campaign_requirements , campaign_discovery_policy , competitor_registry , campaign_competitor_policies , brand_products	F6 — brief parse + eligibility	W2
0006_representations_vectors	creator_representations , creator_content_clusters with VECTOR(384) + HNSW indexes	F5, F7 — semantic retrieval	W3
0007_matching_and_labels	match_scores , match_evidence , eligibility_exclusions , model_versions , pair_labels , experiment_runs	F7, F8, Part 10	W3
0009_wall	wall_posts , wall_comments , wall_reactions , wall_follows + indexes	F13 — the peer community	W5
0008_collaboration_knowledge	Extend proposals with initiated_by ; add proposal_versions , collaborations , agreements , messages , collaboration_outcomes , documents , document_chunks , ai_execution_logs	F10, F11, F12	W4

5.3 Enumerations

Declared once in models/enums.py as str, Enum and used everywhere. The repo has two of these; the other eight are new.

ENUM	VALUES	USED BY
user_role EXISTS	CREATOR · BRAND · ADMIN	user_roles.role
campaign_status	DRAFT · PARSED · READY · PUBLISHED · PAUSED · CLOSED · ARCHIVED	campaigns.status
proposal_status EXTEND	DRAFT · SUBMITTED · REVIEWING · COUNTER_OFFER · ACCEPTED · REJECTED · WITHDRAWN	current enum lacks the middle three
collaboration_status	ACTIVE · IN_PRODUCTION · IN_REVIEW · COMPLETED · CANCELLED	matches the wireframe kanban columns exactly
initiated_by	CREATOR · BRAND	proposals.initiated_by — the two-way switch
platform	YOUTUBE · INSTAGRAM · BLUESKY · MANUAL	social + rate card + content
content_type	REEL · STORY · POST · SHORT · VIDEO · CAROUSEL · OTHER	rate_card_items , campaign formats
data_provenance	CREATOR_PROVIDED · AUTHORIZED_API · PUBLIC_OBSERVABLE · AI_DERIVED · SYNTHETIC	every evidence row; the research filter
discovery_mode	CONVENTIONAL · ADJACENT · FRESH_TO_VERTICAL · NO_PREFERENCE	campaign_discovery_policy.mode
eligibility_status	PASS · FAIL · UNKNOWN · REVIEW_REQUIRED	match_scores.eligibility_status

5.4 The breaking change: dual-role identity

DO THIS FIRST, IN WEEKEND 1, BEFORE ANYONE WRITES A PROFILE ENDPOINT

Your users table has role as a single non-null enum column, and User declares one creator_profile and one brand_profile relationship. The product requires Aarav Mehta to run Aurum Foods in brand mode and hold a creator profile — wireframe 10 shows exactly this. Changing it later means rewriting every auth dependency and every ownership check written before it.

```
def upgrade() -> None:
    op.create_table(
        "user_roles",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("user_id", postgresql.UUID(as_uuid=True), nullable=False),
        sa.Column("role", sa.Enum("CREATOR", "BRAND", "ADMIN",
                                   name="user_role", native_enum=False), nullable=False),
        sa.Column("activated_at", sa.DateTime(timezone=True),
                  nullable=False, server_default=sa.func.now()),
        sa.ForeignKeyConstraint(["user_id"], ["users.id"], ondelete="CASCADE"),
        sa.UniqueConstraint("user_id", "role", name="uq_user_role"),
    )
    op.create_index("ix_user_roles_user_id", "user_roles", ["user_id"])

    # carry every existing account across without losing its role
    op.execute("""
        INSERT INTO user_roles (id, user_id, role, activated_at)
        SELECT gen_random_uuid(), id, role, created_at FROM users
    """)
    op.drop_column("users", "role")
    op.add_column("users", sa.Column("status", sa.String(20),
                                     nullable=False, server_default="ACTIVE"))
```

The ORM side becomes a collection, and the two profile relationships stay exactly as they are — they were already one-to-one on `user_id`, which is correct.

```
class User(TimestampMixin, Base):
    __tablename__ = "users"
    id: Mapped[UUID] = mapped_column(PostgresUUID(as_uuid=True),
                                     primary_key=True, default=uuid4)
    email: Mapped[str] = mapped_column(String(255), unique=True,
                                       index=True, nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="ACTIVE")

    roles: Mapped[list["UserRoleRow"]] = relationship(
        back_populates="user", cascade="all, delete-orphan")
    creator_profile: Mapped["CreatorProfile | None"] = relationship(back_populates="user")
    brand_profile: Mapped["BrandProfile | None"] = relationship(back_populates="user")

    def has_role(self, role: UserRole) -> bool:
        return any(r.role is role for r in self.roles)
```

5.5 Identity and creator tables

DDL – CREATOR PROFILE, PREFERENCES, RATE CARD, AVAILABILITY (MIGRATION 0003)

```
-- EXTEND the existing creator_profiles table; do not recreate it.
ALTER TABLE creator_profiles
  ADD COLUMN country_code      CHAR(2),
  ADD COLUMN city              VARCHAR(120),
  ADD COLUMN languages         JSONB NOT NULL DEFAULT '[]',
  ADD COLUMN content_topics    JSONB NOT NULL DEFAULT '[]',
  ADD COLUMN content_formats   JSONB NOT NULL DEFAULT '[]',
  ADD COLUMN travel_willingness BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN profile_image_url VARCHAR(500),
  ADD COLUMN headline          VARCHAR(160),
  ADD COLUMN is_discoverable   BOOLEAN NOT NULL DEFAULT true,
  ADD COLUMN completeness_score NUMERIC(4,3) NOT NULL DEFAULT 0,
  ADD COLUMN data_provenance    VARCHAR(24) NOT NULL DEFAULT 'CREATOR_PROVIDED',
  ADD COLUMN profile_version   INTEGER NOT NULL DEFAULT 1;

CREATE INDEX ix_creator_country      ON creator_profiles (country_code);
CREATE INDEX ix_creator_categories  ON creator_profiles USING GIN (categories);
CREATE INDEX ix_creator_discover   ON creator_profiles (is_discoverable)
  WHERE is_discoverable = true;

-- Creator-side suitability inputs. Separate table: these drive S_creator, and
-- keeping them apart from descriptive profile fields makes that boundary explicit.
CREATE TABLE creator_preferences (
  creator_id      UUID PRIMARY KEY REFERENCES creator_profiles(id) ON DELETE CASCADE,
  accepted_categories JSONB NOT NULL DEFAULT '[]', -- ['fitness','food']
  excluded_categories JSONB NOT NULL DEFAULT '[]', -- ['alcohol','gambling'] HARD
  excluded_brand_ids  JSONB NOT NULL DEFAULT '[]', -- explicit brand blocks   HARD
  preferred_models    JSONB NOT NULL DEFAULT '[]', -- ['paid','barter','affiliate']
  min_rate_inr        NUMERIC(12,2),             -- HARD floor
  values_statement     TEXT,
  content_restrictions JSONB NOT NULL DEFAULT '[]', -- ['no_medical_claims']
  max_campaigns_month SMALLINT,
  responds_within_hours SMALLINT,
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE rate_card_items (
  id            UUID PRIMARY KEY,
  creator_id    UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  platform      VARCHAR(16) NOT NULL,      -- platform enum
  content_type  VARCHAR(16) NOT NULL,      -- content_type enum
  rate_amount   NUMERIC(12,2) NOT NULL CHECK (rate_amount >= 0),
  currency      CHAR(3) NOT NULL DEFAULT 'INR',
  is_negotiable BOOLEAN NOT NULL DEFAULT true,
  active        BOOLEAN NOT NULL DEFAULT true,
  effective_from DATE NOT NULL DEFAULT CURRENT_DATE,
  effective_to   DATE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- exactly one live price per creator/platform/format
CREATE UNIQUE INDEX uq_rate_active ON rate_card_items (creator_id, platform, content_type)
  WHERE active = true;

CREATE TABLE creator_availability (
  id            UUID PRIMARY KEY,
  creator_id    UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  start_date    DATE NOT NULL,
  end_date      DATE NOT NULL,
  status        VARCHAR(16) NOT NULL DEFAULT 'AVAILABLE', -- AVAILABLE | BLOCKED
  notes         VARCHAR(240),
  CHECK (end_date >= start_date)
);
```

5.6 Social evidence tables

These three tables are the destination for the collectors you have already written. The `transformed_creator.sample.json` contract in your repo maps onto them almost one to one — the mapping is spelled out under the DDL.

```

CREATE TABLE social_accounts (
  id                UUID PRIMARY KEY,
  creator_id        UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  platform          VARCHAR(16) NOT NULL,
  external_account_id VARCHAR(128) NOT NULL,
  handle            VARCHAR(120),
  profile_url       VARCHAR(500),
  account_type      VARCHAR(24),          -- creator | business | personal
  verified_by_oauth BOOLEAN NOT NULL DEFAULT false,
  token_ciphertext  BYTEA,                -- encrypted; NEVER leaves the backend
  granted_scopes     JSONB NOT NULL DEFAULT '[]',
  token_expires_at  TIMESTAMPTZ,
  last_synced_at    TIMESTAMPTZ,
  sync_status       VARCHAR(16) NOT NULL DEFAULT 'NEVER', -- NEVER|OK|STALE|FAILED
  data_provenance   VARCHAR(24) NOT NULL,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (platform, external_account_id)
);
CREATE INDEX ix_social_creator ON social_accounts (creator_id);

-- append-only time series: growth and freshness both become measurable
CREATE TABLE social_metric_snapshots (
  id                UUID PRIMARY KEY,
  social_account_id UUID NOT NULL REFERENCES social_accounts(id) ON DELETE CASCADE,
  captured_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  followers         INTEGER,
  total_content     INTEGER,
  total_views       BIGINT,
  sample_size       SMALLINT,             -- posts the sample was computed over
  sample_window_start TIMESTAMPTZ,
  sample_window_end TIMESTAMPTZ,
  average_views     NUMERIC(14,2),
  median_views      NUMERIC(14,2),
  average_likes     NUMERIC(14,2),
  average_comments  NUMERIC(14,2),
  engagement_rate   NUMERIC(6,4),        -- fraction, not percent
  posts_last_30_days SMALLINT,
  platform_specific JSONB NOT NULL DEFAULT '{}',
  data_provenance   VARCHAR(24) NOT NULL
);
CREATE INDEX ix_snap_account_time ON social_metric_snapshots (social_account_id, captured_at DESC);

CREATE TABLE creator_content_items (
  id                UUID PRIMARY KEY,
  creator_id        UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  social_account_id UUID REFERENCES social_accounts(id) ON DELETE SET NULL,
  external_content_id VARCHAR(160),
  media_type        VARCHAR(16),
  caption           TEXT,
  transcript         TEXT,
  published_at      TIMESTAMPTZ,
  organic_or_sponsored VARCHAR(12) NOT NULL DEFAULT 'UNKNOWN', -- ORGANIC|SPONSORED|UNKNOWN
  brand_entity       VARCHAR(160),      -- resolved sponsor, when declared
  detection_evidence TEXT,              -- why we think it is sponsored
  detection_confidence NUMERIC(4,3),
  metrics_json       JSONB NOT NULL DEFAULT '{}',
  data_provenance   VARCHAR(24) NOT NULL
);
CREATE INDEX ix_content_creator_pub ON creator_content_items (creator_id, published_at DESC);

```

MAPPING YOUR EXISTING SAMPLE CONTRACT ONTO THESE TABLES

`creator.*` → `creator_profiles`. `platform_accounts[]` → one `social_accounts` row each, with `public_metrics` + `sample_metrics` + `platform_specific` becoming one `social_metric_snapshots` row per collection.
`commercial.rate_card` → one `rate_card_items` row per key. `commercial.restricted_brand_categories` → `creator_preferences.excluded_categories`. `audience.*` → `creator_profiles.audience_summary` JSONB.
`data_quality.*` and `provenance.*` → `creator_profiles.completeness_score` plus the `data_provenance` column on each row. `matching_features.*` is *derived* — it belongs in `creator_representations`, not in a stored profile column.

5.7 The normalised social contract

Every adapter — YouTube, Instagram, Bluesky, and any future platform — returns this and only this. Nothing above the integration layer ever sees a provider-shaped field name. This is the single most important piece of decoupling in the system, because provider APIs will change during the project and this contract will not.

```

class NormalisedMetrics(BaseModel):
    followers: int | None = None
    total_content: int | None = None
    total_views: int | None = None
    sample_size: int | None = None
    sample_window_start: datetime | None = None
    sample_window_end: datetime | None = None
    average_views: float | None = None
    median_views: float | None = None
    average_likes: float | None = None
    average_comments: float | None = None
    engagement_rate: float | None = None # fraction 0-1, never a percent
    posts_last_30_days: int | None = None
    platform_specific: dict[str, Any] = {}

class NormalisedAccount(BaseModel):
    platform: Platform
    external_account_id: str
    handle: str | None = None
    profile_url: str | None = None
    account_type: str | None = None
    verified_by_oauth: bool = False
    metrics: NormalisedMetrics
    recent_content: list[NormalisedContent] = []
    captured_at: datetime
    provenance: DataProvenance
    degraded_reason: str | None = None # set when the provider was partly unavailable

class SocialAdapter(Protocol):
    platform: ClassVar[Platform]
    def authorize_url(self, state: str) -> str: ...
    async def exchange_code(self, code: str) -> OAuthTokens: ...
    async def fetch_public(self, handle: str) -> NormalisedAccount: ...
    async def fetch_authorized(self, tokens: OAuthTokens) -> NormalisedAccount: ...

```

TWO RULES THAT WILL SAVE YOU A WEEK

Nulls are information, not zeros. Bluesky has no view count; Instagram hides some insights; a YouTube channel can hide its subscriber count. Store `NULL` and let the feature builder emit a `has_<field>` missingness indicator. Coercing to `0` teaches the ranker that a hidden count means an unpopular creator.

Engagement rate is a fraction everywhere. Your sample file stores `engagement_rate_percent: 5.86`; the technical guide's match response stores `0.058`. Pick the fraction, convert once at the adapter boundary, and format as a percent only in the React component.

5.8 Campaign and constraint tables

DDL – CAMPAIGN LIFECYCLE AND CONFIRMED REQUIREMENTS (MIGRATION 0005)

```
ALTER TABLE campaigns
  ADD COLUMN raw_brief      TEXT,                -- exactly what the brand typed
  ADD COLUMN objective      VARCHAR(32),          -- AWARENESS|NICHE_CREDIBILITY|
                                                -- DIRECT_RESPONSE|UGC

  ADD COLUMN status        VARCHAR(16) NOT NULL DEFAULT 'DRAFT',
  ADD COLUMN budget_min    NUMERIC(12,2),
  ADD COLUMN budget_max    NUMERIC(12,2),
  ADD COLUMN currency      CHAR(3) NOT NULL DEFAULT 'INR',
  ADD COLUMN starts_on     DATE,
  ADD COLUMN ends_on       DATE,
  ADD COLUMN parser_version VARCHAR(32),          -- which prompt produced the draft
  ADD COLUMN parsed_at     TIMESTAMPTZ,
  ADD COLUMN confirmed_at   TIMESTAMPTZ,          -- NULL ⇒ never used for eligibility
  ADD COLUMN published_at   TIMESTAMPTZ;

CREATE INDEX ix_campaign_status ON campaigns (status, published_at DESC);

-- One row per campaign. These are the CONFIRMED values. The LLM proposes;
-- this table only ever receives what a human approved.
CREATE TABLE campaign_requirements (
  campaign_id      UUID PRIMARY KEY REFERENCES campaigns(id) ON DELETE CASCADE,
  category         VARCHAR(64),
  target_age_min   SMALLINT,
  target_age_max   SMALLINT,
  geographies      JSONB NOT NULL DEFAULT '[]',    -- ['IN'] or ['IN-MH']  HARD
  languages        JSONB NOT NULL DEFAULT '[]',
  platforms        JSONB NOT NULL DEFAULT '[]',    -- HARD
  content_types    JSONB NOT NULL DEFAULT '[]',    -- HARD
  preferred_categories JSONB NOT NULL DEFAULT '[]', -- soft: feeds scoring
  mandatory_constraints JSONB NOT NULL DEFAULT '[]', -- ['NO_UNsupported_MEDICAL_CLAIMS']
  min_followers    INTEGER,
  max_rate_inr     NUMERIC(12,2),                  -- HARD
  physical_attendance BOOLEAN NOT NULL DEFAULT false,
  attendance_city   VARCHAR(120),
  extraction_confidence JSONB NOT NULL DEFAULT '{}', -- per-field LLM confidence
  confirmed_by_user_id UUID REFERENCES users(id),
  confirmed_at      TIMESTAMPTZ
);

CREATE TABLE campaign_discovery_policy (
  campaign_id      UUID PRIMARY KEY REFERENCES campaigns(id) ON DELETE CASCADE,
  mode             VARCHAR(24) NOT NULL DEFAULT 'NO_PREFERENCE',
  vertical_experience VARCHAR(16) NOT NULL DEFAULT 'PREFERRED',
  novelty_weight    NUMERIC(4,3) NOT NULL DEFAULT 0.000,
  min_transferable_fit NUMERIC(4,3) NOT NULL DEFAULT 0.620, -- the exploration floor
  enable_fairness_rerank BOOLEAN NOT NULL DEFAULT false,
  diversity_lambda   NUMERIC(4,3) NOT NULL DEFAULT 0.300
);

CREATE TABLE competitor_registry (
  id              UUID PRIMARY KEY,
  parent_entity   VARCHAR(160) NOT NULL,
  brand           VARCHAR(160) NOT NULL,
  product_line    VARCHAR(160),
  aliases         JSONB NOT NULL DEFAULT '[]',
  handles         JSONB NOT NULL DEFAULT '[]'
);

CREATE TABLE campaign_competitor_policies (
  id              UUID PRIMARY KEY,
  campaign_id     UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
  competitor_id   UUID NOT NULL REFERENCES competitor_registry(id),
  association_type VARCHAR(32) NOT NULL DEFAULT 'PAID_ONLY',
  lookback_days   SMALLINT NOT NULL DEFAULT 180,
  action          VARCHAR(16) NOT NULL DEFAULT 'EXCLUDE' -- EXCLUDE | REVIEW
);
```

5.9 Intelligence tables – where explainability lives

`match_scores` is the most important table in the project. The research contribution, the “Why this creator?” panel, the ablation study and the fairness audit all read from it. Write it before you return a response, never after.

```

CREATE TABLE creator_representations (
  id                UUID PRIMARY KEY,
  creator_id        UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  representation_version VARCHAR(32) NOT NULL, -- 'bge-small-en-v1.5/2026-10-11'
  encoder_name      VARCHAR(80) NOT NULL,
  source_text       TEXT, -- exactly what was embedded
  overall_embedding VECTOR(384) NOT NULL,
  topic_metadata    JSONB NOT NULL DEFAULT '{}',
  style_metadata    JSONB NOT NULL DEFAULT '{}',
  value_metadata    JSONB NOT NULL DEFAULT '{}',
  is_current        BOOLEAN NOT NULL DEFAULT true,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_repr_current ON creator_representations (creator_id)
  WHERE is_current = true;
CREATE INDEX ix_repr_vec ON creator_representations
  USING hnsw (overall_embedding vector_cosine_ops);

-- multi-vertical: one creator, several content clusters (Creator Data Strategy §4)
CREATE TABLE creator_content_clusters (
  id                UUID PRIMARY KEY,
  representation_id UUID NOT NULL REFERENCES creator_representations(id) ON DELETE CASCADE,
  label            VARCHAR(80) NOT NULL, -- 'beauty' | 'fashion' | 'travel'
  share           NUMERIC(5,4) NOT NULL, -- 0.42
  embedding        VECTOR(384) NOT NULL,
  item_count       SMALLINT
);
CREATE INDEX ix_cluster_vec ON creator_content_clusters
  USING hnsw (embedding vector_cosine_ops);

CREATE TABLE match_scores (
  id                UUID PRIMARY KEY,
  campaign_id       UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
  creator_id        UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  direction         VARCHAR(16) NOT NULL, -- BRAND_TO_CREATOR | CREATOR_TO_BRAND
  eligibility_status VARCHAR(20) NOT NULL,
  -- the five numbers the UI is allowed to show
  structured_score  NUMERIC(6,5),
  semantic_score    NUMERIC(6,5),
  brand_side_score  NUMERIC(6,5),
  creator_side_score NUMERIC(6,5),
  reciprocal_score  NUMERIC(6,5),
  reranked_score    NUMERIC(6,5),
  final_rank        SMALLINT,
  -- reproducibility
  feature_version   VARCHAR(32) NOT NULL,
  model_version     VARCHAR(32) NOT NULL,
  representation_version VARCHAR(32),
  scoring_profile    VARCHAR(32) NOT NULL, -- which objective weights were used
  components        JSONB NOT NULL DEFAULT '{}', -- every named sub-score
  computed_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (campaign_id, creator_id, direction, model_version, feature_version)
);
CREATE INDEX ix_match_rank ON match_scores (campaign_id, direction, final_rank);
CREATE INDEX ix_match_creator ON match_scores (creator_id, direction, reciprocal_score DESC);

-- the facts the LLM narrator is allowed to see, and nothing else
CREATE TABLE match_evidence (
  id                UUID PRIMARY KEY,
  match_score_id    UUID NOT NULL REFERENCES match_scores(id) ON DELETE CASCADE,
  kind             VARCHAR(32) NOT NULL, -- CATEGORY|AUDIENCE|BUDGET|ENGAGEMENT|
  -- SEMANTIC|PREFERENCE|FRESHNESS|WARNING
  label            VARCHAR(120) NOT NULL,
  value_text       VARCHAR(240),
  value_numeric    NUMERIC(14,4),
  provenance       VARCHAR(24),
  captured_at      TIMESTAMPTZ
);

-- why a creator never reached ranking. Auditable, and it is a research metric.
CREATE TABLE eligibility_exclusions (
  id                UUID PRIMARY KEY,
  campaign_id       UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
  creator_id        UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  rule_code         VARCHAR(48) NOT NULL, -- GEOGRAPHY | PLATFORM | BUDGET |
  -- CREATOR_EXCLUSION | COMPETITOR | AVAILABILITY
  rule_version      VARCHAR(16) NOT NULL,
  detail            VARCHAR(240),
  confidence        NUMERIC(4,3),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE model_versions (
  id                UUID PRIMARY KEY,
  model_version     VARCHAR(32) UNIQUE NOT NULL,
  algorithm         VARCHAR(40) NOT NULL, -- logreg | random_forest | xgboost
  feature_version   VARCHAR(32) NOT NULL,

```

```
trained_at    TIMESTAMPTZ NOT NULL,  
train_rows    INTEGER,  
metrics       JSONB NOT NULL DEFAULT '{}', -- {"ndcg@5":0.71,"p@5":0.64}  
artifact_path VARCHAR(300),  
is_active     BOOLEAN NOT NULL DEFAULT false  
);
```

5.10 Collaboration and knowledge tables

DDL – THE TWO-WAY PROPOSAL MODEL AND RAG STORE (MIGRATION 0008)

```
-- ONE table serves both directions. This is the core two-way design decision:
-- a brand invitation and a creator proposal are the same row with a different
-- initiated_by value, so every downstream screen, state transition and metric
-- is written once instead of twice.
ALTER TABLE proposals
  ADD COLUMN initiated_by      VARCHAR(8) NOT NULL DEFAULT 'CREATOR', -- CREATOR|BRAND
  ADD COLUMN match_score_id    UUID REFERENCES match_scores(id),      -- the ranking it came from
  ADD COLUMN deliverables      JSONB NOT NULL DEFAULT '[]',
  ADD COLUMN currency          CHAR(3) NOT NULL DEFAULT 'INR',
  ADD COLUMN collaboration_model VARCHAR(16) NOT NULL DEFAULT 'PAID', -- PAID|BARTER|HYBRID
  ADD COLUMN available_from    DATE,
  ADD COLUMN current_version   SMALLINT NOT NULL DEFAULT 1,
  ADD COLUMN ai_assisted       BOOLEAN NOT NULL DEFAULT false,
  ADD COLUMN responded_at      TIMESTAMPTZ;
CREATE INDEX ix_proposal_campaign ON proposals (campaign_id, status);
CREATE INDEX ix_proposal_creator  ON proposals (creator_profile_id, status);

-- every counter-offer appends; commercial history is never overwritten
CREATE TABLE proposal_versions (
  id          UUID PRIMARY KEY,
  proposal_id UUID NOT NULL REFERENCES proposals(id) ON DELETE CASCADE,
  version_no   SMALLINT NOT NULL,
  amount       NUMERIC(12,2),
  currency     CHAR(3) NOT NULL DEFAULT 'INR',
  deliverables JSONB NOT NULL DEFAULT '[]',
  message      TEXT,
  created_by_user_id UUID NOT NULL REFERENCES users(id),
  created_by_role   VARCHAR(8) NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (proposal_id, version_no)
);

-- created ONLY by an explicit human ACCEPT
CREATE TABLE collaborations (
  id          UUID PRIMARY KEY,
  accepted_proposal_id UUID UNIQUE NOT NULL REFERENCES proposals(id),
  campaign_id  UUID NOT NULL REFERENCES campaigns(id),
  creator_id   UUID NOT NULL REFERENCES creator_profiles(id),
  brand_id     UUID NOT NULL REFERENCES brand_profiles(id),
  status       VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  agreed_amount NUMERIC(12,2),
  currency     CHAR(3) NOT NULL DEFAULT 'INR',
  starts_on    DATE, ends_on DATE,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE agreements (
  id          UUID PRIMARY KEY,
  collaboration_id UUID NOT NULL REFERENCES collaborations(id) ON DELETE CASCADE,
  version      SMALLINT NOT NULL DEFAULT 1,
  draft_text   TEXT NOT NULL,
  evidence_refs JSONB NOT NULL DEFAULT '[]', -- chunk ids + proposal version
  generated_by VARCHAR(16) NOT NULL DEFAULT 'AI_DRAFT',
  status       VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  brand_approved_at TIMESTAMPTZ,
  creator_approved_at TIMESTAMPTZ,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE messages (
  id          UUID PRIMARY KEY,
  collaboration_id UUID REFERENCES collaborations(id) ON DELETE CASCADE,
  proposal_id  UUID REFERENCES proposals(id) ON DELETE CASCADE,
  sender_user_id UUID NOT NULL REFERENCES users(id),
  body        TEXT NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (collaboration_id IS NOT NULL OR proposal_id IS NOT NULL)
);

CREATE TABLE documents (
  id          UUID PRIMARY KEY,
  owner_type  VARCHAR(12) NOT NULL, -- BRAND | CAMPAIGN | CREATOR
  owner_id    UUID NOT NULL,
  document_type VARCHAR(32) NOT NULL, -- BRAND_GUIDELINE | CAMPAIGN_BRIEF | PORTFOLIO
  filename    VARCHAR(255) NOT NULL,
  storage_uri  VARCHAR(500) NOT NULL,
  mime_type   VARCHAR(80),
  checksum    VARCHAR(64),
  page_count  SMALLINT,
  ingest_status VARCHAR(16) NOT NULL DEFAULT 'PENDING',
  uploaded_by UUID NOT NULL REFERENCES users(id),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```

CREATE TABLE document_chunks (
  id          UUID PRIMARY KEY,
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
  chunk_index SMALLINT NOT NULL,
  chunk_text  TEXT NOT NULL,
  page_number SMALLINT,
  embedding   VECTOR(384) NOT NULL,
  token_count SMALLINT,
  metadata    JSONB NOT NULL DEFAULT '{}',
  UNIQUE (document_id, chunk_index)
);

CREATE INDEX ix_chunk_vec ON document_chunks USING hnsw (embedding vector_cosine_ops);

CREATE TABLE ai_execution_logs (
  id          UUID PRIMARY KEY,
  workflow_type VARCHAR(40) NOT NULL,      -- BRIEF_PARSE | EXPLAIN | PROPOSAL_DRAFT |
                                           -- RAG_QUERY | MATCH_GRAPH
  subject_type VARCHAR(24), subject_id UUID,
  actor_user_id UUID REFERENCES users(id),
  provider     VARCHAR(40), model VARCHAR(80),
  prompt_version VARCHAR(32), state_version VARCHAR(32),
  latency_ms   INTEGER, estimated_cost NUMERIC(10,5),
  input_tokens INTEGER, output_tokens INTEGER,
  status       VARCHAR(16) NOT NULL,      -- OK | FAILED | FALLBACK
  error_category VARCHAR(40),
  correlation_id VARCHAR(64),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

5.11 Research tables

DDL — LABELS AND EXPERIMENT TRACKING

```

CREATE TABLE pair_labels (
  id          UUID PRIMARY KEY,
  campaign_id UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
  creator_id  UUID NOT NULL REFERENCES creator_profiles(id) ON DELETE CASCADE,
  rater_id    VARCHAR(40) NOT NULL,      -- which teammate or rubric persona
  brand_rating SMALLINT CHECK (brand_rating BETWEEN 0 AND 3),
  creator_rating SMALLINT CHECK (creator_rating BETWEEN 0 AND 3),
  mutually_relevant BOOLEAN GENERATED ALWAYS AS
    (brand_rating >= 2 AND creator_rating >= 2) STORED,
  rationale   TEXT,
  label_source VARCHAR(24) NOT NULL,      -- HUMAN | RUBRIC | SYNTHETIC
  rubric_version VARCHAR(16) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (campaign_id, creator_id, rater_id, rubric_version)
);

CREATE TABLE experiment_runs (
  id          UUID PRIMARY KEY,
  run_name    VARCHAR(80) NOT NULL,      -- 'E4-hybrid-onesided'
  variant     VARCHAR(48) NOT NULL,      -- popularity|rules|semantic|hybrid|
                                           -- reciprocal|reciprocal+rerank
  dataset_version VARCHAR(32) NOT NULL,
  feature_version VARCHAR(32), model_version VARCHAR(32),
  split_strategy VARCHAR(32),      -- campaign_holdout | creator_holdout
  metrics      JSONB NOT NULL DEFAULT '{}',
  notes       TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE collaboration_outcomes (
  id          UUID PRIMARY KEY,
  proposal_id UUID UNIQUE NOT NULL REFERENCES proposals(id) ON DELETE CASCADE,
  accepted    BOOLEAN, completed BOOLEAN,
  brand_rating SMALLINT, creator_rating SMALLINT,
  outcome_metrics JSONB NOT NULL DEFAULT '{}',
  recorded_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

WHAT COLLABORATION_OUTCOMES IS FOR — AND WHAT IT IS NOT

Collect it, do not train on it. With a prototype you will have single-digit accepted collaborations. Any model trained on that is noise presented as intelligence, and the plan documents say so explicitly. It exists so the demo can show a closed loop and so future work has a schema to fill.

5.12 The Wall — community tables

The Wall is where brands and creators meet each other before a campaign exists: ideas, open calls, results and questions, with follows and threaded replies. It is a genuine product surface and it is also the answer to “how does anyone arrive on this platform in the first place?” — which neither the matching engine nor the campaign flow answers.

THE TWO RULES THAT MAKE THE WALL SAFE TO BUILD

1. Wall content never enters the ranking pipeline. No post text is embedded into `creator_representations`, no engagement count becomes a feature, and `wall_*` tables are unreachable from `services/discovery.py`. Enforce it with an import test. If posting could move a creator up a shortlist, every score in the evaluation becomes contestable and the research contribution is gone.

2. A thread converts into an ordinary proposal. The “+ Collab” action on a post creates a normal `proposals` row with the thread attached as context — it does not create a parallel negotiation path. One state machine, one audit trail, one set of metrics.

DDL — WALL POSTS, COMMENTS, REACTIONS, FOLLOWS (MIGRATION 0009)

```
CREATE TABLE wall_posts (
  id          UUID PRIMARY KEY,
  author_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  author_role  VARCHAR(8) NOT NULL,      -- BRAND | CREATOR — which profile posted
  author_profile_id UUID NOT NULL,      -- brand_profiles.id or creator_profiles.id
  post_kind    VARCHAR(16) NOT NULL DEFAULT 'IDEA',
                                -- IDEA | OPEN_CALL | RESULT | QUESTION
  body        TEXT NOT NULL CHECK (char_length(body) <= 2000),
  topics      JSONB NOT NULL DEFAULT '[]', -- ['sustainability','reels']
  media_uri    VARCHAR(500),             -- optional single image
  linked_campaign_id UUID REFERENCES campaigns(id) ON DELETE SET NULL,
  visibility   VARCHAR(16) NOT NULL DEFAULT 'MEMBERS', -- MEMBERS | FOLLOWERS
  status       VARCHAR(16) NOT NULL DEFAULT 'PUBLISHED', -- PUBLISHED|HIDDEN|REMOVED
  reply_count  INTEGER NOT NULL DEFAULT 0, -- denormalised counters, updated in the
  reaction_count INTEGER NOT NULL DEFAULT 0, -- same transaction as the child insert
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  edited_at    TIMESTAMPTZ
);
CREATE INDEX ix_wall_feed    ON wall_posts (status, created_at DESC);
CREATE INDEX ix_wall_author  ON wall_posts (author_profile_id, created_at DESC);
CREATE INDEX ix_wall_topics  ON wall_posts USING GIN (topics);

CREATE TABLE wall_comments (
  id          UUID PRIMARY KEY,
  post_id     UUID NOT NULL REFERENCES wall_posts(id) ON DELETE CASCADE,
  author_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  author_role  VARCHAR(8) NOT NULL,
  body        TEXT NOT NULL CHECK (char_length(body) <= 1000),
  status       VARCHAR(16) NOT NULL DEFAULT 'PUBLISHED',
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_wall_comment_post ON wall_comments (post_id, created_at);

CREATE TABLE wall_reactions (
  post_id     UUID NOT NULL REFERENCES wall_posts(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (post_id, user_id)      -- one reaction per person; the PK is the guard
);

CREATE TABLE wall_follows (
  follower_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  followed_profile_id UUID NOT NULL, -- a brand_profiles.id or creator_profiles.id
  followed_kind VARCHAR(8) NOT NULL, -- BRAND | CREATOR
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (follower_user_id, followed_profile_id)
);
CREATE INDEX ix_follow_profile ON wall_follows (followed_profile_id);
```

SCOPE DISCIPLINE — WHAT THE WALL IS NOT

No direct messaging, no reposts with commentary, no notifications service, no feed-ranking model, no image processing beyond storing one file, no automated moderation. The feed is `ORDER BY created_at DESC` over posts from people you follow plus posts in topics you follow, and that is enough. Reports set `status = HIDDEN` and land in an admin list a human reads. Everything past that is future work, and saying so is a better answer than a half-built recommendation feed.

API catalogue

About sixty endpoints under `/api/v1`. Auth column: **Public** needs nothing, **Auth** needs a valid token, **Brand/Creator** additionally require that role to be active and to own the resource.

6.1 Conventions

- Resource paths for CRUD; an explicit verb sub-path only where a state transition or an AI operation happens — `/parse-brief`, `/confirm-requirements`, `/accept`.
- Every error is `{"code", "message", "details", "correlation_id"}`. One `DomainError` hierarchy in `core/errors.py`, one exception handler in `main.py`.
- Lists take `?limit=&offset=` and return `{"items":[], "total":n, "limit":1, "offset":o}`. Pick offset paging and never mix it with cursors.
- The active role travels in an `X-Active-Role` header, validated against `user_roles` on every mutating call. Never trust a role sent in a body.
- Anything that may exceed ~2 s returns `202` with a `run_id`, and the client polls `/ai/runs/{id}`.

6.2 Identity, profiles and commercial terms

METHOD	PATH	AUTH	DOES	WRITES	WK
POST	<code>/auth/register</code>	Public	Create account, activate one or both roles	<code>users</code> , <code>user_roles</code>	1
POST	<code>/auth/login</code>	Public	Access + refresh token	—	1
POST	<code>/auth/refresh</code>	Auth	Rotate access token	—	1
GET	<code>/me</code>	Auth	Identity + active roles + which profiles exist	—	1
POST	<code>/me/roles</code>	Auth	Activate the second professional profile later	<code>user_roles</code>	1
POST	<code>/creators</code>	Creator	Create creator profile	<code>creator_profiles</code>	2
GET / PATCH	<code>/creators/{id}</code>	Auth / Creator	Read or update profile	<code>creator_profiles</code>	2
GET	<code>/creators</code>	Auth	Filtered directory search (non-AI)	—	2
GET / PUT	<code>/creators/{id}/preferences</code>	Creator	Accepted, excluded, min rate, values	<code>creator_preferences</code>	2
GET / PUT	<code>/creators/{id}/rate-card</code>	Creator	Replace the live rate card as a set	<code>rate_card_items</code>	2
GET / PUT	<code>/creators/{id}/availability</code>	Creator	Date windows	<code>creator_availability</code>	2
POST	<code>/creators/{id}/representation/rebuild</code>	Creator	Re-embed after a profile change	<code>creator_representations</code>	3
POST / GET / PATCH	<code>/brands</code> · <code>/brands/{id}</code>	Brand	Brand profile	<code>brand_profiles</code>	2
GET / POST	<code>/brands/{id}/products</code>	Brand	Products under a brand	<code>brand_products</code>	4

6.3 Campaigns and brief extraction

METHOD	PATH	AUTH	DOES	WRITES	WK
POST	<code>/campaigns</code>	Brand	Create with <code>raw_brief</code> ; status DRAFT	<code>campaigns</code>	2
GET	<code>/campaigns</code>	Auth	Brand mode: own campaigns. Creator mode: published, filterable.	—	2
GET / PATCH	<code>/campaigns/{id}</code>	Auth / Brand	Detail, including requirements when confirmed	<code>campaigns</code>	2
POST	<code>/campaigns/{id}/parse-brief</code>	Brand	LLM → draft structure + per-field confidence. Never writes requirements.	<code>campaigns.parsed_at</code> , <code>ai_execution_logs</code>	5
POST	<code>/campaigns/{id}/confirm-requirements</code>	Brand	Persist the human-approved structure	<code>campaign_requirements</code>	2
PUT	<code>/campaigns/{id}/discovery-policy</code>	Brand	Mode, novelty, fairness toggle	<code>campaign_discovery_policy</code>	6
PUT	<code>/campaigns/{id}/competitor-policy</code>	Brand	Competitor exclusions	<code>campaign_competitor_policies</code>	4
POST	<code>/campaigns/{id}/publish</code>	Brand	Validate then publish; makes it creator-discoverable	<code>campaigns.status</code>	2

6.4 Discovery – both directions

METHOD	PATH	AUTH	DIRECTION	DOES	WK
POST	/campaigns/{id}/recommendations/generate	Brand	B→C	Run the full pipeline, persist <code>match_scores</code> , return top-K	3
GET	/campaigns/{id}/recommendations	Brand	B→C	Read persisted ranking; <code>?include_excluded=true</code> shows the filtered set with reasons	3
GET	/creators/{id}/recommendations	Creator	C→B	Ranked <i>campaigns</i> for this creator, creator-side utility leading	4
POST	/creators/{id}/recommendations/refresh	Creator	C→B	Recompute against all published campaigns	4
GET	/matches/{campaign_id}/{creator_id}	Auth	both	All components + evidence + grounded explanation. Response is role-shaped.	3
POST	/matches/{campaign_id}/{creator_id}/explain	Auth	both	Generate the narrative from stored evidence only	5
GET	/campaigns/{id}/exclusions	Brand	B→C	Audit view of who was filtered and by which rule	3

6.5 Collaboration – one state machine, two entry points

METHOD	PATH	AUTH	DOES	SETS	WK
POST	/campaigns/{id}/invitations	Brand	Brand invites a creator	<code>initiated_by=BRAND</code>	4
POST	/campaigns/{id}/proposals	Creator	Creator pitches a campaign	<code>initiated_by=CREATOR</code>	4
GET	/proposals	Auth	Role-aware inbox/outbox: <code>?box=incoming outgoing&status=</code>	—	4
GET	/proposals/{id}	Auth	Detail with full version history	—	4
POST	/proposals/{id}/counter	Auth	Either side counters; appends a version	<code>COUNTER_OFFER</code>	4
POST	/proposals/{id}/accept	Auth	The only path that creates a collaboration	<code>ACCEPTED</code> + <code>collaborations</code>	4
POST	/proposals/{id}/reject · /withdraw	Auth	Terminal transitions	<code>REJECTED</code> / <code>WITHDRAWN</code>	4
GET	/collaborations	Auth	Kanban data grouped by status	—	4
GET / PATCH	/collaborations/{id}	Auth	Detail; advance status	<code>collaborations</code>	4
POST	/collaborations/{id}/messages	Auth	Simple threaded note	<code>messages</code>	6
POST	/collaborations/{id}/agreement/draft	Auth	AI drafts terms from approved evidence	<code>agreements</code>	6
POST	/collaborations/{id}/agreement/approve	Auth	One side approves; both required	<code>agreements.*_approved_at</code>	6

6.6 Knowledge, integrations and operations

METHOD	PATH	AUTH	DOES	WK
POST	/brands/{id}/documents · /campaigns/{id}/documents	Brand	Upload PDF; queue chunk + embed as a background task	5
GET	/documents/{id}	Auth	Metadata + ingest status + chunk count	5
POST	/campaigns/{id}/knowledge/query	Auth	Grounded Q&A: answer + source chunks + page numbers	5
GET	/integrations/{platform}/connect	Creator	Signed <code>state</code> + provider authorize URL	3
GET	/integrations/{platform}/callback	Public	Verify state, exchange code, store encrypted token	3
POST	/integrations/{platform}/{account_id}/sync	Creator	Refresh the normalised snapshot	3
DELETE	/integrations/{platform}/{account_id}	Creator	Revoke and delete the token; keep past snapshots	3
GET	/integrations/youtube/public-lookup?handle=	Auth	Key-only channel lookup — the no-OAuth fallback	3
POST	/ai/proposals/draft	Auth	Editable proposal text from stored evidence	5
GET	/ai/runs/{id}	Auth	Status, latency, prompt version, fallback flag	5
GET	/admin/experiments	Admin	Variant comparison table for the report	7

6.7 The Wall

METHOD	PATH	AUTH	DOES	WRITES	WK
GET	/wall	Auth	Feed: followed profiles + followed topics, newest first. ?kind=&topic=&cursor=	—	5
POST	/wall/posts	Auth	Publish as the active profile; author_role comes from the header, never the body	wall_posts	5
GET	/wall/posts/{id}	Auth	Thread: post plus comments	—	5
DELETE	/wall/posts/{id}	Author	Soft delete → status = REMOVED	wall_posts	5
POST	/wall/posts/{id}/comments	Auth	Reply; increments reply_count in the same transaction	wall_comments	5
PUT / DELETE	/wall/posts/{id}/reaction	Auth	Idempotent on the composite primary key	wall_reactions	5
PUT / DELETE	/wall/follows/{profile_id}	Auth	Follow or unfollow a brand or creator	wall_follows	5
POST	/wall/posts/{id}/report	Auth	Flag for the admin queue; sets HIDDEN above a threshold	wall_posts	6
POST	/wall/posts/{id}/to-proposal	Auth	The bridge. Creates a normal proposal with the thread as context — no parallel negotiation path	proposals	6

Flow-by-flow build guide

Twelve flows, in build order. Each one gives the screen, the exact sequence from click to SQL, the request and response bodies, the failure behaviour, and what “done” means. Build them in this order — each depends on the one before.

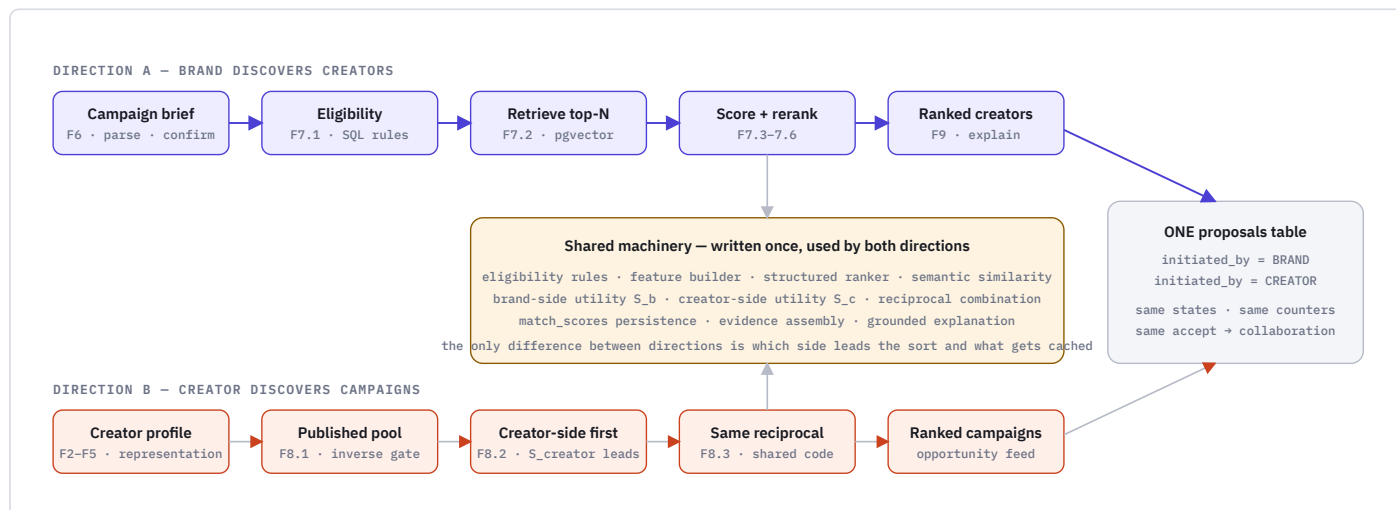


Figure 4 — The two-way design in one picture. Direction B is not a second recommender: it reuses every component and changes only the query subject, the leading utility and the cache key. Build direction A first, then B costs roughly one weekend.

SCREEN Wireframe 03 — “Welcome to BrandBridge AI”, role cards, *Create one account*

TABLES `users`, `user_roles`

FILES `endpoints/auth.py` · `services/identity.py` · `repositories/user_repo.py` · `api/deps.py` · `core/security.py`

BLOCKS Every other flow

- Form submits** `{email, password, roles: ["BRAND", "CREATOR"]}`. At least one role is required; both is legal and is the wireframe's headline case.
- Router** `POST /api/v1/auth/register` validates with `RegisterRequest`. Password rule: minimum 10 characters, enforced in the Pydantic validator so the message is identical in the OpenAPI schema and the UI.
- Service** `identity.register()` calls `user_repo.get_by_email()`. On a hit, raise `ConflictError` → HTTP 409. Do not leak whether an email exists on *login*; on register the conflict is unavoidable and acceptable.
- Hash** with the existing `hash_password()`. It is already PBKDF2-SHA256 at 210k iterations — do not replace it.
- Insert** one `users` row and one `user_roles` row per requested role, in one transaction. The unique constraint `(user_id, role)` makes double activation a no-op error rather than a duplicate.
- Issue tokens.** Access token claims: `{sub: user_id, roles: ["BRAND", "CREATOR"], exp}`, 30 minutes. Refresh token: 7 days, `{sub, type: "refresh"}`. Roles live in the claim so `require_role` does not hit the database on every request — but ownership checks always do.
- Respond 201** with tokens plus `{profiles: {brand: false, creator: false}}`. The client reads this and routes straight into whichever profile-builder is missing.
- Client** stores the token in the zustand store, sets `activeRole` to the first role, and every request from here carries `Authorization: Bearer` and `X-Active-Role`.

API/DEPS.PY — THE THREE DEPENDENCIES EVERYTHING ELSE USES

```
def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    payload = decode_token(token) # raises AuthError on bad/expired
    user = user_repo.get(db, UUID(payload["sub"]))
    if user is None or user.status != "ACTIVE":
        raise AuthError("inactive_account")
    return user

def get_active_role(
    x_active_role: Annotated[str | None, Header()] = None,
    user: User = Depends(get_current_user),
) -> UserRole:
    roles = [r.role for r in user.roles]
    if x_active_role is None:
        return roles[0]
    role = UserRole(x_active_role)
    if role not in roles:
        raise ForbiddenError("role_not_activated") # → 403
    return role

def require_role(required: UserRole):
    def guard(role: UserRole = Depends(get_active_role)) -> UserRole:
        if role is not required:
            raise ForbiddenError(f"requires_{required.value.lower()}_mode")
        return role
    return guard

# usage: @router.post("/campaigns", dependencies=[Depends(require_role(UserRole.BRAND))])
```

Tests. Register with both roles → two `user_roles` rows. Login with a wrong password → 401 with a generic message. Call a brand endpoint with `X-Active-Role: CREATOR` → 403. Expired token → 401, not 500.

Done when a single account can hold both roles, switch between them, and be refused access to the other mode's endpoints.

This is the flow worth reading closely, because every other write flow in the system follows the same eight-layer shape. Once you can build this one, `brand_profiles`, `campaigns` and `proposals` are the same work with different columns.

SCREEN Three-step wizard: Identity → Audience & content → Commercial terms

ENDPOINT `POST /api/v1/creators`, then `PATCH /api/v1/creators/{id}`

TABLES `creator_profiles` now; `creator_preferences`, `rate_card_items`, `creator_availability` in F3

Which field goes where

FORM FIELD	COLUMN	TYPE	RULE	USED
Display name	<code>display_name</code>	VARCHAR(255)	required, 2–80 chars	UI
Headline	<code>headline</code>	VARCHAR(160)	optional	UI, embed text
Bio	<code>bio</code>	TEXT	≤ 2000 chars	embed → semantic score
Country	<code>country_code</code>	CHAR(2)	ISO-3166 alpha-2, uppercased	hard filter
City	<code>city</code>	VARCHAR(120)	optional	hard filter when campaign needs attend
Languages	<code>languages</code>	JSONB	ISO-639-1 array, ≥1	hard filter
Categories	<code>categories</code>	JSONB	1–5 from a fixed vocabulary	structured feature, GIN prefilter
Content topics	<code>content_topics</code>	JSONB	free text, ≤10	embed
Content formats	<code>content_formats</code>	JSONB	<code>content_type</code> values	hard filter
Audience	<code>audience_summary</code>	JSONB	<code>{age_range, primary_country, distribution, source, measured_at}</code>	audience fit feature
Willing to travel	<code>travel_willingness</code>	BOOLEAN	default false	hard filter
Discoverable	<code>is_discoverable</code>	BOOLEAN	default true	retrieve gate
— computed —	<code>completeness_score</code>	NUMERIC(4,3)	service-computed, never client-sent	UI nuclear cold-start feature
— computed —	<code>data_provenance</code>	VARCHAR(24)	<code>CREATOR_PROVIDED</code> on this path	research filter

The eight layers, in order

1 Client. `react-hook-form` + a `zod` schema that mirrors the Pydantic model. Submit through `createCreatorProfile()` in `services/creators.ts`.

2 Router — `backend/app/api/v1/endpoints/creators.py`. Replace the placeholder entirely:

```
@router.post("/creators", response_model=CreatorProfileOut, status_code=201,
            dependencies=[Depends(require_role(UserRole.CREATOR))])
def create_creator_profile(
    payload: CreatorProfileCreate,
    db: Session = Depends(get_db),
    user: User = Depends(get_current_user),
) -> CreatorProfileOut:
    profile = creator_service.create_profile(db, user_id=user.id, data=payload)
    return CreatorProfileOut.model_validate(profile)
```

Five lines. No session use, no query, no business rule. That is the whole point of the layer contract.

3 **Schema** — `schemas/creator.py`. `CreatorProfileCreate` holds only what a client may send; `completeness_score`, `data_provenance`, `profile_version` and `user_id` are deliberately absent. `country_code` gets a validator that uppercases and checks length; `categories` gets one that rejects values outside the vocabulary.

4 **Service** — `services/creator.py`. This is where the rules live:

```
def create_profile(db: Session, *, user_id: UUID, data: CreatorProfileCreate) -> CreatorProfile:
    if creator_repo.get_by_user(db, user_id):
        raise ConflictError("creator_profile_exists")          # → 409

    profile = CreatorProfile(
        id=uuid4(), user_id=user_id,
        **data.model_dump(exclude_unset=True),
        data_provenance=DataProvenance.CREATOR_PROVIDED,
        profile_version=1,
    )
    profile.completeness_score = compute_completeness(profile) # pure function, unit-tested
    creator_repo.add(db, profile)
    db.commit()                                               # service owns the boundary
    db.refresh(profile)

    if profile.completeness_score >= 0.6:                      # enough text to embed
        background.enqueue(rebuild_representation, profile.id) # F5, never blocks the response
    return profile
```

5 **Repository** — `repositories/creator_repo.py`. Three functions: `get_by_user`, `add`, `update`. They call `db.add()` and `db.execute(select(...))` and nothing else. No `commit` here — the service owns it, which is what makes multi-table writes atomic later.

6 **Database**. One INSERT into `creator_profiles`. `user_id` is UNIQUE, so a race produces an `IntegrityError` you catch and re-raise as the same `ConflictError` — the constraint is the real guard, the pre-check is only for a better message.

7 **Response 201**. `CreatorProfileOut` with the id, the stored values, `completeness_score`, and `missing_fields` so the UI can render “add a rate card to appear in budget-filtered searches”.

8 **Client cache**. `queryClient.invalidateQueries(['me'])` and `['creator', id]`. Navigate to step 2 of the wizard.

REQUEST · POST /API/V1/CREATORS

```
{
  "display_name": "Aarav Fitness",
  "headline": "Strength training and honest nutrition for Indian audiences",
  "bio": "I create fitness, workout and healthy nutrition content for young Indian audiences interested in strength training and healthy lifestyles.",
  "country_code": "IN",
  "city": "Mumbai",
  "languages": ["en", "hi"],
  "categories": ["fitness", "nutrition"],
  "content_topics": ["strength training", "healthy recipes", "weight management"],
  "content_formats": ["REEL", "SHORT", "VIDEO"],
  "audience_summary": {
    "age_range": "18-30",
    "primary_country": "IN",
    "country_distribution": {"IN": 82.0, "AE": 5.0, "US": 4.0},
    "source": "creator_provided",
    "measured_at": "2026-09-01T00:00:00Z"
  },
  "travel_willingness": true
}
```

RESPONSE · 201 CREATED

```
{
  "id": "8f2c...",
  "user_id": "1a9d...",
  "display_name": "Aarav Fitness",
  "country_code": "IN",
  "categories": ["fitness", "nutrition"],
  "completeness_score": 0.72,
  "data_provenance": "CREATOR_PROVIDED",
  "is_discoverable": true,
  "profile_version": 1,
  "missing_fields": ["rate_card", "preferences", "social_account"],
  "created_at": "2026-10-03T09:14:22Z"
}
```

It is a weighted count of populated fields — bio 0.15, categories 0.15, languages 0.10, audience 0.15, rate card 0.20, preferences 0.15, a connected social account 0.10. It drives the UI nudge, but it is also a *cold-start feature*: a new creator with a complete profile should outrank a new creator with an empty one, and the ranker needs that signal as a number. If the client could send it, it would be gameable and useless as a feature.

Tests. Happy path 201. Second profile for the same user → 409. Brand mode → 403. Empty `languages` → 422 naming the field. `compute_completeness` unit-tested against six fixture profiles with no database.

F3 Rate card, preferences and availability

WEEKEND 2

These three tables carry more weight than their size suggests: `min_rate_inr` and `excluded_categories` are **hard filters**, and the whole creator-side utility S_c is computed from `creator_preferences`. Without this flow, direction B has nothing to rank on.

- 1 Rate card is replace-as-a-set.** `PUT /creators/{id}/rate-card` takes the full list. The service marks existing rows `active=false`, `effective_to=today` and inserts new ones. History survives, the partial unique index guarantees one live price per platform+format, and a brand's budget filter is never ambiguous.
- 2 Preferences are upsert.** `PUT /creators/{id}/preferences` writes the single `creator_preferences` row. Validate that `accepted_categories` and `excluded_categories` do not intersect → 422.
- 3 Mark the hard fields in the UI.** A small lock icon beside `min_rate_inr`, `excluded_categories` and `excluded_brand_ids`, with the text “campaigns that break this will never reach you”. This is a product feature and it is also how you explain deterministic filtering in the viva.
- 4 Recompute completeness** and re-enqueue the representation rebuild, because `values_statement` is part of the embedded text.

REQUEST · `PUT /API/V1/CREATORS/{ID}/RATE-CARD`

```
{ "currency": "INR",
  "items": [
    { "platform": "INSTAGRAM", "content_type": "REEL", "rate_amount": 7000, "is_negotiable": true },
    { "platform": "INSTAGRAM", "content_type": "STORY", "rate_amount": 2500, "is_negotiable": true },
    { "platform": "YOUTUBE", "content_type": "SHORT", "rate_amount": 9000, "is_negotiable": false },
    { "platform": "YOUTUBE", "content_type": "VIDEO", "rate_amount": 25000, "is_negotiable": true }
  ]
}
```

Done when a campaign with `max_rate_inr = 8000` requesting a YouTube SHORT excludes this creator, and `eligibility_exclusions` records `rule_code = BUDGET` with the detail `"9000 > 8000"`.

REUSES

Your existing `youtubedatacollector.py`, `instagramcollector*.py`, `bskydatacollector.py` — refactored, not rewritten

TABLES

`social_accounts`, `social_metric_snapshots`, `creator_content_items`, `integration_sync_logs`

- 1 **Connect.** `GET /integrations/youtube/connect` generates a signed, single-use `state` containing `{user_id, platform, nonce, exp}`, stores the nonce, and returns the provider authorize URL. The frontend redirects; it never builds the URL itself and never sees a client secret.
- 2 **Callback.** `GET /integrations/youtube/callback?code=&state=` verifies the state signature and nonce, exchanges the code server-side, and encrypts the refresh token with `OAuth_TOKEN_ENCRYPTION_KEY` before it touches a column. The access token is never persisted in plaintext and never returned to React. Assert this in a test that greps the response body.
- 3 **First sync** runs immediately as a background task: `adapter.fetch_authorized()` → `NormalisedAccount` → one `social_accounts` upsert, one `social_metric_snapshots` insert, up to 50 `creator_content_items` inserts.
- 4 **Sponsored vs. organic.** For each content item, run a deterministic classifier over the caption: paid-partnership markers, `#ad`, `#sponsored`, `#collab`, and brand handles present in `competitor_registry`. Store the matched span in `detection_evidence` and a confidence. **An organic mention is never automatically a paid association** — that safeguard is in the plan and it is what keeps competitor exclusion defensible.
- 5 **Freshness.** Every sync writes `last_synced_at` and a row in `integration_sync_logs` with request class, success or failure, and any quota error. The UI shows “metrics as of 16 Sep” wherever a number appears.
- 6 **The three fallbacks**, in order: authorized OAuth → public lookup by handle with an API key only → manual entry by the creator, stored as `CREATOR_PROVIDED`. All three write the same tables and the same normalised contract; only `data_provenance` and `verified_by_oauth` differ.

DEMO-DAY INSURANCE

Bluesky needs no key and no app review, so wire it first and keep it as the live-integration demo. YouTube public lookup needs only an API key. Meta's Instagram path needs a Professional account linked to a Facebook Page plus app review, and it is the one most likely to be blocked when you need it — so show the POC, seed the database from your committed `data/raw/` snapshots, and never make the demo depend on a live Meta call.

This turns a creator row into the vectors that make semantic retrieval possible. It runs as a background task after profile, preference or content changes — never inside a request.

- 1 **Assemble the source text** deterministically and store it in `creator_representations.source_text`, so any embedding can be reproduced later:

```
parts = [
    profile.headline or "",
    profile.bio or "",
    " ".join(profile.categories),
    " ".join(profile.content_topics),
    " ".join(profile.languages),
    prefs.values_statement or "",
    " ".join(c.caption[:280] for c in recent_content[:30] if c.caption),
]
source_text = " \n".join(p for p in parts if p.strip())
```

- 2 **Encode** with a cached singleton `SentenceTransformer("BAAI/bge-small-en-v1.5")` → 384 dimensions. Normalise to unit length so cosine distance and inner product agree.
Load the model once at import in `ai/embeddings/encoder.py`. Loading it per request adds several seconds and will look like a backend bug.
- 3 **Cluster for multi-vertical representation.** Embed each content item, run KMeans with `k = min(4, n_items // 8)`, label each cluster by its nearest category term, and write one `creator_content_clusters` row per cluster with its `share`. This is the Creator Data Strategy's "Beauty 42% / Fashion 28% / Travel 18%" model, and it is a named ablation in Part 10.
- 4 **Version and swap atomically.** Insert the new representation with `is_current=false`, then in one transaction set the old row false and the new row true. The partial unique index enforces exactly one current row per creator.
- 5 **Never delete old representations.** A `match_scores` row from week 4 references `representation_version`; deleting it destroys the ability to reproduce that ranking.

SCREEN

Wireframe 06 — three steps: Brief → AI structure → Invite

TABLES

campaigns, campaign_requirements, ai_execution_logs

BUILD THE MANUAL FORM FIRST

In weekend 2, ship `POST /campaigns` + `POST /confirm-requirements` with a plain form. The LLM parser in weekend 5 becomes a *pre-fill* on top of a flow that already works. Do it in the other order and an LLM outage in November takes the campaign feature with it.

- Create.** `POST /campaigns` with `{name, raw_brief, objective}`. Status `DRAFT`. The raw text is stored verbatim and is never modified again — the extraction evaluation in Part 10 needs the original input.
- Parse.** `POST /campaigns/{id}/parse-brief`. The service calls `ai/llm/structured.py`, which passes the `CampaignRequirementsDraft` Pydantic schema to the provider as a constrained output schema. On a schema violation, retry once with the validation error appended; on a second failure, return `parse_failed` and let the brand fill the form. **This endpoint never writes `campaign_requirements`.**
- Confidence per field.** The prompt requires a confidence per extracted field. Anything under 0.7 renders with an amber border and the label “please check”. Store the whole map in `campaign_requirements.extraction_confidence` — it is a research artifact, not just UI polish.
- Confirm.** `POST /campaigns/{id}/confirm-requirements` writes the row the brand approved, stamps `confirmed_by_user_id` and `confirmed_at`. **Eligibility reads only confirmed rows.** An unconfirmed LLM guess never becomes a hard constraint — this is the single rule that makes the whole parsing feature safe.
- Publish.** `POST /campaigns/{id}/publish` validates that requirements exist, at least one platform and one content type are set, and a budget is present. Sets `PUBLISHED` and `published_at`, which is what makes the campaign visible to direction B.
- Log the run** in `ai_execution_logs`: provider, model, prompt version, latency, token counts, status. Every LLM call in this system logs a row; that table is where the “AI/ML in Practice” latency and cost numbers come from.

POST /API/V1/CAMPAIGNS/{ID}/PARSE-BRIEF — REQUEST AND RESPONSE

```
// request
{ "brief": "High-protein millet snack. India. Fitness creators. Reels and Shorts.
  Budget INR 50K. No unsupported medical claims." }

// 200 OK — a DRAFT. requires_confirmation is always true.
{
  "campaign_id": "c31f...",
  "parser_version": "campaign-parser-v1",
  "requires_confirmation": true,
  "structured": {
    "category": "health_fitness",
    "geographies": ["IN"],
    "budget_max": 50000,
    "currency": "INR",
    "platforms": ["INSTAGRAM", "YOUTUBE"],
    "content_types": ["REEL", "SHORT"],
    "preferred_categories": ["fitness", "nutrition"],
    "mandatory_constraints": ["NO_UNSUPPORTED_MEDICAL_CLAIMS"],
    "target_age_min": null,
    "target_age_max": null
  },
  "confidence": {
    "category": 0.94, "geographies": 0.97, "budget_max": 0.91,
    "platforms": 0.96, "content_types": 0.93,
    "preferred_categories": 0.81,
    "mandatory_constraints": 0.88,
    "target_age_min": 0.0, "target_age_max": 0.0
  },
  "unmapped_text": ["millet"],
  "run_id": "run_7c2a..."
}
```

Note the nulls. The brief never mentioned an age range, so the parser returns `null` at confidence 0. It does not guess “18–35” because that is the plausible answer. A guessed hard constraint silently removes eligible creators and nobody ever finds out — missing is the correct output, and the UI asks.

The central flow of the project. Six stages, each independently testable, each writing something you can audit. Build stages 1–3 in weekend 3 with a rule-based score, then swap in the trained model in weekend 5 without changing a single endpoint.

ENDPOINT `POST /api/v1/campaigns/{id}/recommendations/generate`

SERVICE `services/discovery.py :: generate_for_campaign()`

WRITES `match_scores`, `match_evidence`, `eligibility_exclusions`

Stage 1 — deterministic eligibility SQL ONLY, NO MODEL

Runs in the database, returns ids, and writes one exclusion row per rejected creator with the rule that rejected them. Nothing that fails here can be rescued by a high similarity score — that is the whole point.

SERVICES/ELIGIBILITY.PY — THE SIX RULES

```
RULE_VERSION = "eligibility-v1"

def eligible_creator_ids(db, req: CampaignRequirements, pol: CompetitorPolicy) -> EligibilityResult:
    """Returns passing ids plus a reason for every exclusion. Pure SQL + Python set logic."""

    # R1 GEOGRAPHY      creator.country_code must be in req.geographies
    # R2 PLATFORM       creator must have an active rate_card_item on a required platform
    # R3 CONTENT_TYPE   creator.content_formats ≥ at least one req.content_types
    # R4 LANGUAGE       creator.languages n req.languages ≠ 0 (skipped when req empty)
    # R5 BUDGET         min(active rate for required platform+type) ≤ req.max_rate_inr
    #                  AND creator_preferences.min_rate_inr ≤ req.max_rate_inr
    # R6 CREATOR_BLOCK  campaign.category ∉ prefs.excluded_categories
    #                  AND brand_id ∉ prefs.excluded_brand_ids
    # R7 AVAILABILITY   only when req.physical_attendance or campaign dates are set
    # R8 COMPETITOR     resolve creator_content_items.brand_entity against
    #                  competitor_registry within lookback_days.
    #                  verified paid association → FAIL
    #                  ambiguous / organic / unresolved → REVIEW_REQUIRED, never FAIL

    stmt = (
        select(CreatorProfile.id)
        .where(CreatorProfile.is_discoverable.is_(True))
        .where(CreatorProfile.country_code.in_(req.geographies))          # R1
        .where(CreatorProfile.languages.op("!=")(req.languages))          # R4 JSONB any-key
        .where(exists().where(and_(
            RateCardItem.creator_id == CreatorProfile.id,
            RateCardItem.active.is_(True),
            RateCardItem.platform.in_(req.platforms),
            RateCardItem.content_type.in_(req.content_types),
            RateCardItem.rate_amount <= req.max_rate_inr)))                # R2 + R3 + R5
    )
    passing = set(db.scalars(stmt))
    # R6–R8 evaluated in Python so each exclusion carries a readable reason
    ...
    return EligibilityResult(passing=passing, exclusions=exclusions,
                             review_required=review, rule_version=RULE_VERSION)
```

WHY EXCLUSIONS GET THEIR OWN TABLE

Three reasons, in increasing order of importance. It powers the “show me who you filtered out” audit screen. It makes a brand trust the system, because an empty result set with reasons is debuggable and an empty result set without them looks broken. And **rule accuracy and false-exclusion rate are evaluation metrics** in the research plan — you cannot report them if you never stored the decisions.

Stage 2 — semantic candidate retrieval PGVECTOR

- 1 Embed the campaign query text: `raw_brief + category + preferred_categories + objective`, with the same encoder and the same normalisation as F5. A mismatched encoder here silently produces meaningless similarities.
- 2 ANN search restricted to the eligible set, returning N = 50–100:

```
SELECT r.creator_id,
       1 - (r.overall_embedding <=> :q) AS semantic_score
FROM   creator_representations r
WHERE  r.is_current = true
       AND r.creator_id = ANY(:eligible_ids)
ORDER  BY r.overall_embedding <=> :q      -- cosine distance, uses the HNSW index
LIMIT  :n;
```

`<=>` is cosine distance, so smaller is closer and `1 - distance` is the similarity you store. Order by the distance, not by the subtraction, or the index is not used.

3 **Discovery mode shapes the pool.** `CONVENTIONAL` keeps same-vertical hits. `ADJACENT` also queries `creator_content_clusters` so a creator whose *secondary* cluster matches surfaces. `FRESH_TO_VERTICAL` selects creators with no sponsored history in this category but a transferable fit above `min_transferable_fit` — the exploration floor that stops novelty from becoming irrelevance.

4 Tag each candidate with its pool so the UI and the metrics can separate them: `same_vertical`, `adjacent`, `fresh_to_vertical`.

Stage 3 — feature assembly `ONE CONTRACT`

One function, one version string, used identically by training and by inference. If these two ever diverge, every metric you report becomes fiction — so they call the same code.

ML/FEATURES/BUILDER.PY

```
FEATURE_VERSION = "feat-v1"

def build_pair_features(campaign: CampaignBundle, creator: CreatorBundle) -> dict[str, float]:
    latest = creator.latest_snapshot          # may be None → missingness flags fire
    return {
        # profile fit
        "category_overlap": jaccard(campaign.preferred_categories, creator.categories),
        "topic_overlap": jaccard(campaign.topics, creator.content_topics),
        # audience fit
        "age_overlap": interval_overlap(campaign.age_range, creator.age_range),
        "country_match": float(creator.country_code in campaign.geographies),
        "language_overlap": jaccard(campaign.languages, creator.languages),
        # channel fit
        "platform_coverage": coverage(campaign.platforms, creator.active_platforms),
        "format_coverage": coverage(campaign.content_types, creator.content_formats),
        # commercial fit
        "budget_headroom": clamp((campaign.max_rate - creator.min_quote)
                                  / max(campaign.max_rate, 1.0), -1, 1),
        "rate_ratio": safe_div(creator.min_quote, campaign.max_rate),
        "model_compatibility": float(campaign.model in creator.preferred_models),
        # performance
        "engagement_rate": latest.engagement_rate if latest else 0.0,
        "log_followers": log1p(latest.followers) if latest else 0.0,
        "views_consistency": safe_div(latest.median_views, latest.average_views) if latest else 0.0,
        "posting_frequency": latest.posts_last_30_days if latest else 0.0,
        # semantic
        "semantic_overall": creator.semantic_score,
        "semantic_best_cluster": creator.best_cluster_score,
        # reciprocity inputs
        "category_accepted": float(campaign.category in creator.accepted_categories),
        "meets_min_rate": float(campaign.max_rate >= (creator.min_rate or 0)),
        # trust
        "profile_completeness": creator.completeness_score,
        "verified_account": float(creator.has_oauth_account),
        # missingness — information, not noise
        "has_verified_audience": float(creator.audience_source == "AUTHORIZED_API"),
        "has_api_metrics": float(latest is not None),
        "metric_age_days": days_since(latest.captured_at) if latest else 999.0,
        "has_collab_history": float(creator.past_collaborations > 0),
    }
```

Stage 4 — scoring both sides, then combining

The academic core. Two utilities are computed independently and only then combined.

QUANTITY	HOW	INPUTS
S_{ml} structured	Active model from <code>model_versions</code> : LR → RF → XGBoost as the ladder progresses	the feature dict above
S_{sem} semantic	Cosine similarity, already computed in stage 2	campaign vector, creator vector
S_b brand-side	Weighted blend under a scoring profile chosen by <code>campaign.objective</code>	S_{ml} , S_{sem} , engagement, budget fit, safety
S_c creator-side	Transparent weighted rule score — interpretable by design, because a creator must be able to read why	category accepted, rate headroom, model match, values, workload, audience relevance
S_r reciprocal	Harmonic mean as the default; arithmetic, weighted and <code>min()</code> as the compared alternatives	S_b , S_c

```

SCORING_PROFILES = {
    # hypotheses to validate, NOT facts
    "AWARENESS": {"m1":.30,"sem":.20,"reach":.30,"engagement":.10,"budget":.10},
    "NICHE_CREDIBILITY": {"m1":.30,"sem":.30,"reach":.05,"engagement":.25,"budget":.10},
    "DIRECT_RESPONSE": {"m1":.40,"sem":.15,"reach":.10,"engagement":.20,"budget":.15},
    "UGC": {"m1":.25,"sem":.30,"reach":.05,"engagement":.15,"budget":.25},
}

def reciprocal(s_brand: float, s_creator: float, method: str = "harmonic") -> float:
    if method == "harmonic":
        if s_brand <= 0 or s_creator <= 0: return 0.0
        return 2 * s_brand * s_creator / (s_brand + s_creator)
    if method == "arithmetic": return (s_brand + s_creator) / 2
    if method == "min": return min(s_brand, s_creator)
    if method == "weighted": return 0.6 * s_brand + 0.4 * s_creator
    raise ValueError(method)

```

THE EXAMPLE THAT CARRIES THE WHOLE ARGUMENT

Pair A scores $S_b = 0.95$, $S_c = 0.20$. Pair B scores 0.76 and 0.74. A conventional brand-side ranker puts A first and the creator declines, because the campaign pays below their floor and sits outside their accepted categories. Harmonic mean gives A = 0.33 and B = 0.75, so B ranks first — and B is the collaboration that actually happens. That inversion is your research contribution, it is one chart, and it belongs on the first slide of the final presentation.

Report the weights above as *hypotheses under test*. Compare fixed universal weights against objective-aware weights and publish the difference. Presenting hand-set business weights as learned intelligence is the criticism the plan documents warn about most often.

Stage 5 — fairness and diversity re-ranking

- 1 Off by default, controlled per campaign by `campaign_discovery_policy.enable_fairness_rerank`. Always report the paired result — with and without — because the trade-off is the finding.
- 2 MMR over the top 30: `score = \lambda \cdot S_r - (1-\lambda) \cdot \max \text{similarity to already-selected}`, where similarity combines the creator vector and the follower-size band.
- 3 **A relevance floor is mandatory.** No creator below `min_transferable_fit` enters the list for diversity's sake. Diversity is not fairness, and forcing an irrelevant creator in front of a brand harms the creator too.
- 4 Store both `reciprocal_score` and `reranked_score` in the same row, so “relevance lost to re-ranking” is a subtraction rather than a re-run.

Stage 6 — persist, then return

One transaction: up to N `match_scores` rows with every component and all three version strings, the `match_evidence` rows behind the top K, and the exclusion rows from stage 1. Only then build the HTTP response — and build it *from what you wrote*, never from the in-memory objects. That guarantees the screen and the audit table can never disagree.

```

{
  "campaign_id": "c31f...",
  "generated_at": "2026-10-18T11:02:44Z",
  "pipeline": {
    "eligible": 248, "excluded": 92, "review_required": 7,
    "retrieved": 75, "scored": 75, "returned": 10,
    "feature_version": "feat-v1", "model_version": "xgb-2026-10-17",
    "representation_version": "bge-small-en-v1.5/2026-10-11",
    "scoring_profile": "NICHE_CREDIBILITY",
    "reciprocal_method": "harmonic", "rerank_applied": false,
    "latency_ms": 840
  },
  "items": [
    {
      "rank": 1,
      "creator": {
        "id": "8f2c...", "display_name": "Aarav Fitness", "city": "Mumbai",
        "categories": ["fitness", "nutrition"], "followers": 127000,
        "engagement_rate": 0.0573, "pool": "same_vertical"},
      "scores": {
        "structured_ml": 0.89, "semantic": 0.94, "brand_side": 0.92,
        "creator_side": 0.87, "reciprocal": 0.90, "reranked": null},
      "components": {
        "category_overlap": 1.0, "age_overlap": 0.92, "country_match": 1.0,
        "platform_coverage": 1.0, "budget_headroom": 0.28,
        "engagement_rate": 0.0573, "semantic_best_cluster": 0.94,
        "category_accepted": 1.0, "meets_min_rate": 1.0},
      "evidence": [
        {
          "kind": "CATEGORY", "label": "Fitness and nutrition match the brief",
          "value_text": "fitness, nutrition", "provenance": "CREATOR_PROVIDED"},
        {
          "kind": "AUDIENCE", "label": "Audience inside the target age band",
          "value_numeric": 0.92, "provenance": "CREATOR_PROVIDED"},
        {
          "kind": "BUDGET", "label": "Reel rate is within budget",
          "value_text": "INR 7,000 vs INR 8,000 cap", "provenance": "CREATOR_PROVIDED"},
        {
          "kind": "ENGAGEMENT", "label": "Engagement 5.73% across 45 sampled posts",
          "value_numeric": 0.0573, "provenance": "AUTHORIZED_API",
          "captured_at": "2026-09-16T08:00:00Z"},
        {
          "kind": "PREFERENCE", "label": "Creator accepts food and wellness campaigns",
          "provenance": "CREATOR_PROVIDED"},
        {
          "kind": "WARNING", "label": "No prior collaboration history on this platform"}
      ],
      "explanation": null
    }
  ]
}

```

Note that `explanation` is `null`. The narrative is generated on demand by F9, from `evidence` and nothing else. Ranking never waits on an LLM.

The documents you already have specify this as a single endpoint. It deserves more, because it is half of the product and half of the research claim — and because three things genuinely differ from direction A. Everything else is the same code.

ASPECT	DIRECTION A — BRAND → CREATOR	DIRECTION B — CREATOR → BRAND
Query subject	one campaign vector	one creator vector
Search space	eligible creators for this campaign	all <code>PUBLISHED</code> campaigns where this creator passes eligibility
Eligibility direction	campaign requirements applied to creators	same rules, inverted — plus creator-side hard blocks applied first
Leading sort	S_b informs, S_r ranks	S_c gates, S_r ranks — a campaign the creator would refuse never appears, however much the brand would want them
Trigger	brand clicks Generate	background refresh on login, on profile change, and on new campaign publication
Cache key	<code>(campaign_id, model_version)</code>	<code>(creator_id, model_version)</code>
Row written	<code>direction = BRAND_TO_CREATOR</code>	<code>direction = CREATOR_TO_BRAND</code>
Response shape	creator cards with brand-relevant evidence	campaign cards with creator-relevant evidence: pay vs. rate card, deliverables, deadline, brand category

- 1 Creator-side hard blocks run first.** Before any campaign is considered, drop every campaign whose category is in `excluded_categories`, whose brand is in `excluded_brand_ids`, or whose `max_rate_inr` is below the creator's `min_rate_inr`. This is a cheap SQL pass and it is also an ethical position: the creator's stated boundaries are enforced before the system starts persuading them.
- 2 Inverse eligibility.** For each surviving campaign, run the *same* `eligibility` module with the arguments swapped. One rule set, one version string, two call sites — so a rule fix in week 6 cannot make the two directions disagree.
- 3 Retrieve.** Query the creator's current vector against campaign query vectors. Cache `campaign_query_embedding` on the campaign row at publish time so this is a vector-to-vector comparison, not a re-embed per creator.
- 4 Score with the same functions.** `build_pair_features(campaign, creator)` is symmetric — it already takes both bundles. Compute S_b and S_c exactly as in F7 and combine with the same `reciprocal()`.
- 5 Sort by S_r , but gate on S_c .** Drop anything with S_c below 0.5 regardless of S_r . A creator's feed filled with campaigns that are wrong for them is exactly the one-sided failure this project exists to fix.
- 6 Persist with `direction = CREATOR_TO_BRAND`.** The unique constraint includes direction, so both views of the same pair coexist and can be compared — which is itself an interesting result for the report.
- 7 Render the opportunity feed.** Each card leads with what the creator needs to decide: pay against their own rate card, deliverables, deadline, brand category, and a one-line reason. Not a percentage.

```

{
  "creator_id": "8f2c...",
  "refreshed_at": "2026-10-25T07:31:10Z",
  "pipeline": {
    "published_campaigns": 86,
    "passed_creator_blocks": 74,
    "eligible": 31,
    "scored": 31,
    "returned": 8,
    "model_version": "xgb-2026-10-17",
    "gate": "creator_side>=0.5"
  },
  "items": [
    {
      "rank": 1,
      "campaign": {
        "id": "c31f...",
        "name": "Protein Crunch Launch",
        "brand_name": "Aurum Foods",
        "brand_category": "food_wellness",
        "objective": "NICHE_CREDIBILITY",
        "budget_max": 50000,
        "currency": "INR",
        "content_types": ["REEL", "SHORT"],
        "closes_on": "2026-11-14"
      },
      "scores": {
        "creator_side": 0.87,
        "brand_side": 0.92,
        "reciprocal": 0.90
      },
      "creator_view": {
        "pay_vs_rate_card": "INR 18,000 offered · your Reel + 2 Stories = INR 12,000",
        "headroom_pct": 50.0,
        "category_accepted": true,
        "meets_min_rate": true,
        "model_match": "paid — matches your preference",
        "effort": "1 Reel + 2 Stories over 3 weeks",
        "conflicts": []
      },
      "why": "This brand's nutrition focus matches your top content cluster (62% of your recent posts), the budget clears your Reel rate with room to negotiate, and food is on your accepted list.",
      "actions": ["view_campaign", "draft_proposal", "not_interested"]
    }
  ]
}

```

THE “NOT INTERESTED” BUTTON IS NOT COSMETIC

It writes a lightweight negative signal. With a few hundred of them you have the only genuine creator-side preference data the project will ever collect, and it costs one table and one endpoint. Do not model it as an outcome label — log it, report the count, and name it as future work.

- 1 **Assemble the payload deterministically** from `match_scores` and `match_evidence`. The LLM receives a typed `MatchEvidence` object: passed constraints, named component values, semantic similarity, creator-side preference matches, freshness timestamps, and explicit warnings. It never receives the creator row, the campaign row, or a database handle.
- 2 **Prompt contract**, versioned in `ai/prompts/explain_match_v1.md`: use only the supplied facts; never invent a number; state missing data as missing; four sentences maximum; no promise about campaign outcomes.
- 3 **Validate the output** before it reaches the client. Extract every number from the generated text and assert each one appears in the payload. A mismatch is logged as `status=FALLBACK` and the UI renders the structured evidence list instead. This check is cheap, it is testable, and it is the answer to “how do you know it did not hallucinate?”
- 4 **Cache** on `(match_score_id, prompt_version)`. The same match explained twice must produce the same text, or your human-evaluation ratings mean nothing.
- 5 **Always render the structured panel** underneath the prose. The prose is the convenience; the component bars and the evidence chips with their `captured_at` stamps are the actual explanation, and they work with the LLM switched off.

WHAT THE MODEL RECEIVES — NOTHING ELSE

```
{
  "campaign": { "objective": "NICHE_CREDIBILITY", "category": "health_fitness",
    "budget_max": 50000, "content_types": ["REEL", "SHORT"] },
  "passed_constraints": [ "GEOGRAPHY", "PLATFORM", "CONTENT_TYPE", "BUDGET", "CREATOR_BLOCK" ],
  "components": { "category_overlap": 1.0, "age_overlap": 0.92, "platform_coverage": 1.0,
    "budget_headroom": 0.28, "engagement_rate": 0.0573, "semantic_overall": 0.94 },
  "creator_side": { "category_accepted": true, "meets_min_rate": true,
    "model_match": "paid" },
  "freshness": { "metrics_captured_at": "2026-09-16T08:00:00Z", "metric_age_days": 32 },
  "warnings": [ "no_prior_collaboration_history" ],
  "scores": { "brand_side": 0.92, "creator_side": 0.87, "reciprocal": 0.90 }
}
```

One state machine serves both entry points. A brand invitation and a creator proposal are the same row.

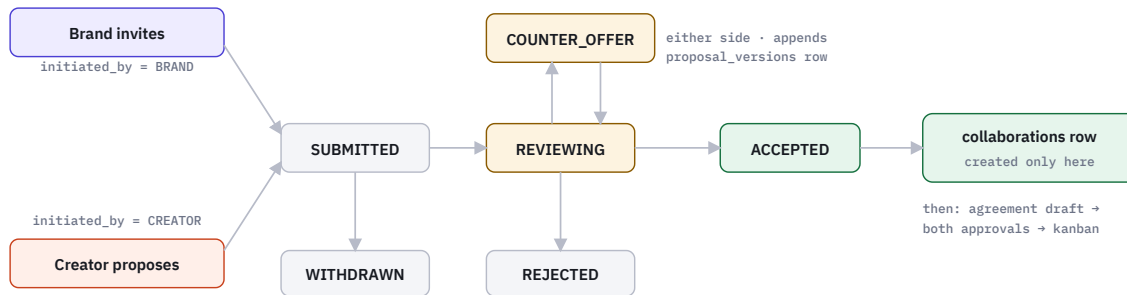


Figure 5 — No transition happens without an authenticated human POST. The AI may draft the message and the agreement text; it never advances a state.

- Create.** `POST /campaigns/{id}/invitations` (brand) or `POST /campaigns/{id}/proposals` (creator) — two routes, one service function, differing only in `initiated_by`. Both store `match_score_id` so you can later ask whether high-ranked pairs converted more often.
- Guard the transitions in the service, not the route.** A single `ALLOWED: dict[Status, set[Status]]` map, plus a check that the actor is the counterparty — you cannot accept your own proposal. Invalid transitions raise `ConflictError` → 409, never a silent no-op.
- Counter appends.** Insert a `proposal_versions` row, increment `current_version`, set status `COUNTER_OFFER`. The previous amount is never updated in place — the negotiation history is the interesting artifact.
- Accept is the only creator of a collaboration.** One transaction: set `ACCEPTED`, insert `collaborations` with the agreed amount copied from the accepted version, and optionally insert a `collaboration_outcomes` stub. The `UNIQUE` on `accepted_proposal_id` makes a double-accept race impossible at the database level.
- Agreement.** `POST /collaborations/{id}/agreement/draft` generates readable terms from the accepted proposal version plus retrieved brand-guideline chunks, and stores `evidence_refs` pointing at those chunks. Status stays `DRAFT` until **both** `brand_approved_at` and `creator_approved_at` are set. Wireframe 08 shows exactly this, including “Awaiting response”.
- Kanban.** `GET /collaborations` groups by `collaboration_status`, whose values were chosen to match the wireframe's columns one to one.

- 1 Upload.** `POST /brands/{id}/documents`, multipart. Validate MIME and a size cap, checksum, write to `DOCUMENT_STORAGE_PATH`, insert a `documents` row with `ingest_status=PENDING`, return 202 immediately.
- 2 Ingest** as a background task: `pypdf` extracts per page → `RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)` → embed each chunk with the same encoder as F5 → insert `document_chunks` with the page number retained. Set `ingest_status=READY`.
Keeping the page number is what lets the UI cite “Brand guidelines, p. 4” instead of an anonymous paragraph. It is the difference between grounded and merely plausible.
- 3 Query.** `POST /campaigns/{id}/knowledge/query`. Embed the question, retrieve top-k=5 chunks filtered by owner so a brand can never retrieve another brand's guidelines, and pass only those chunks to the LLM.
- 4 Answer with sources, or refuse.** If the best similarity is below a threshold, return “not covered in the uploaded documents” rather than letting the model answer from its own knowledge. A confident answer with no source is the failure mode that makes RAG worthless.
- 5 Treat document text as untrusted.** An uploaded PDF may contain text shaped like instructions. Wrap retrieved content in a delimited block, instruct the model that it is reference data only, and never let retrieved text reach a tool-calling context.

RESPONSE · `POST /CAMPAIGNS/{ID}/KNOWLEDGE/QUERY`

```
{
  "question": "Can the creator say this snack helps with weight loss?",
  "answer": "No. The brand guidelines prohibit weight-loss and medical claims, and
    require visible paid-partnership disclosure on every deliverable.",
  "sources": [
    {"document_id": "d90a...", "filename": "aurum-brand-guidelines-2026.pdf",
      "page": 4, "chunk_index": 11, "similarity": 0.87,
      "excerpt": "Creators must not make weight-loss, therapeutic or medical claims..."},
    {"document_id": "d90a...", "filename": "aurum-brand-guidelines-2026.pdf",
      "page": 7, "chunk_index": 19, "similarity": 0.74,
      "excerpt": "All paid collaborations must carry a visible disclosure..."}
  ],
  "grounded": true,
  "run_id": "run_9f14..."
}
```

BUILD THIS LAST, AND ONLY WRAP THINGS THAT ALREADY WORK

Every node below must already exist and pass its own unit tests as a plain function. LangGraph adds stateful sequencing, checkpointing and an explicit human interrupt — it adds nothing if the underlying tools are unfinished, and debugging a broken tool through a graph costs three times as long. If weekend 6 is tight, ship the deterministic service orchestration instead and present the graph as designed-and-scoped. That is an honest engineering decision and it defends well.

- 1 **State.** `CampaignMatchState` as a `TypedDict`: `request_id`, `actor_user_id`, `active_role`, `campaign_id`, `raw_brief`, `confirmed_requirements`, `eligible_creator_ids`, `excluded`, `snapshot_ids`, `feature_version`, `model_version`, `structured_scores`, `semantic_scores`, `creator_side_scores`, `reciprocal_scores`, `retrieved_chunk_ids`, `explanation_objects`, `selected_creator_ids`, `approval_status`, `proposal_draft_id`, `errors`.
- 2 **Nodes, in order.** `parse_campaign` → `validate_requirements` → `search_creators` → `load_cached_metrics` → `calculate_ml_match` → `calculate_semantic_similarity` → `combine_reciprocal` → `generate_explanation` → conditional `retrieve_brand_guidelines` → **HUMAN_APPROVAL** `interrupt` → `generate_proposal_draft` → END.
- 3 **The interrupt is the point.** `interrupt_before=["generate_proposal_draft"]` with a checkpointer. The graph pauses, the brand reviews in the UI, and an explicit resume continues it. That is the human-in-the-loop requirement expressed as architecture rather than as a sentence in a report.
- 4 **Nine tools**, each a thin typed wrapper over an existing service function: `search_creators`, `get_creator_profile`, `get_youtube_statistics`, `get_instagram_statistics`, `calculate_ml_match`, `calculate_semantic_similarity`, `retrieve_brand_guidelines`, `generate_match_explanation`, `generate_collaboration_proposal`. Seven are deterministic or pure retrieval; only two call a language model.
- 5 **Log every transition** to `ai_execution_logs` with the state version, so “correct tool sequence” and “no unauthorised action” become measurable claims rather than assertions.

The only surface in the product that works before a campaign exists. A brand with no campaign yet, or a creator nobody has searched for, still has a reason to open BrandBridge — and the threads that form here are where a measurable number of collaborations will actually start.

SCREENS Wireframes 10 and 11 — community feed, and an idea thread converting into work

TABLES `wall_posts`, `wall_comments`, `wall_reactions`, `wall_follows`

FILES `endpoints/wall.py` · `services/wall.py` · `repositories/wall_repo.py`

MUST NOT Touch `services/discovery.py`, `ml/` or `creator_representations`

- 1 **Post as the active profile.** `POST /wall/posts` reads `author_role` from the `X-Active-Role` header and resolves `author_profile_id` server-side. A dual-role user posting in brand mode appears as Aurum Foods; the same person in creator mode appears as Aarav Fitness. Never accept an author identity from the request body.
- 2 **Four post kinds, because they behave differently.** `OPEN_CALL` is the one that matters commercially — it gets the “+ Collab” affordance and can be filtered on. `IDEA`, `RESULT` and `QUESTION` are community. One enum, one filter chip row, no separate feature.
- 3 **The feed is a boring query.** Posts from profiles you follow, union posts whose `topics` overlap topics you follow, `ORDER BY created_at DESC`, keyset-paginated on `(created_at, id)`. **No ranking model.** Resist this specifically — a second recommender would double the evaluation surface and halve the depth of the one that matters.
- 4 **Counters update in the same transaction** as the comment or reaction insert. `reply_count` and `reaction_count` exist so the feed query never fans out into per-post aggregates at render time.
- 5 **Reactions are idempotent by construction.** `PUT` inserts on the composite primary key and swallows the conflict; `DELETE` removes. No toggle logic in the service, no double-counting on a double tap.
- 6 **The bridge to real work.** `POST /wall/posts/{id}/to-proposal` creates an ordinary `proposals` row with `initiated_by` set from the actor's active role and the thread id stored as context. Everything downstream — counters, acceptance, collaboration, agreement — is the F10 state machine unchanged.
This is the design decision that keeps the Wall from becoming a shadow product. There is exactly one way to agree to work in BrandBridge.
- 7 **Moderation is a human queue.** `POST /report` increments a count; above a threshold the post flips to `HIDDEN` and appears in an admin list. No classifier, no automated takedown, and say so in the report — an honest boundary beats a fragile filter.
- 8 **Treat post text as untrusted** everywhere it is rendered and everywhere it could reach a model. It is user-generated content that a prompt-injection fixture should target in the test suite, exactly like an uploaded PDF.

THE ISOLATION TEST — WRITE IT IN THE SAME PR AS THE FIRST WALL ENDPOINT

A test that fails if any module under `ml/` or `services/discovery.py` imports anything from `wall`. Ten lines with `ast` or a grep in CI. It is what lets you say in the viva, without hesitation, that posting activity cannot influence a match score — and it is the kind of claim an examiner will probe.

Tests. Post in brand mode → author renders as the brand. Double-react → one row. Comment → `reply_count` is 1. Follow then unfollow → feed changes. Convert a thread → exactly one proposal row with the correct `initiated_by`. Import-isolation test passes.

Done when two accounts can follow each other, post, reply, and turn a thread into a proposal that lands in the normal inbox — and the isolation test is green.

End-to-end request and response traces

Two complete journeys, hop by hop, with the file touched, the table read or written, and a latency budget at each step. Use these as integration-test scripts and as the demo-day script.

8.1 Trace A — brand creates a campaign and gets ranked creators

Aurum Foods launches Protein Crunch. Aarav Mehta is signed in, brand mode active. Budget target: ~1.6 s excluding the LLM parse, which is a separate user-initiated step.

1	CLIENT	POST /api/v1/campaigns — <code>{name, raw_brief, objective: "NICHE_CREDIBILITY"}</code> , headers <code>Authorization: Bearer ...</code> , <code>X-Active-Role: BRAND</code> .
2	DEPS	<code>get_current_user</code> decodes the JWT; <code>require_role(BRAND)</code> checks the header against <code>user_roles</code> . ~3 ms
3	SERVICE	<code>campaign.create()</code> resolves the brand profile from <code>user.id</code> , inserts <code>campaigns</code> with <code>status=DRAFT</code> and the brief stored verbatim. 201 with the campaign id. ~25 ms
4	CLIENT	Brand clicks <i>Extract with AI</i> → POST /campaigns/{id}/parse-brief .
5	AI	<code>structured.extract()</code> calls the provider with the <code>CampaignRequirementsDraft</code> schema. Response validated; one retry on a schema error. Writes <code>ai_execution_logs</code> . 1.2–3 s — this is why it is its own step
6	CLIENT	Step 2 renders the draft. Three fields are amber (confidence < 0.7). The brand sets the age band to 18–35 and caps the per-creator rate at INR 8,000.
7	SERVICE	POST /confirm-requirements → insert <code>campaign_requirements</code> with <code>confirmed_by_user_id</code> . Only now do hard constraints exist. ~20 ms
8	SERVICE	POST /publish → validate, set <code>PUBLISHED</code> , cache <code>campaign_query_embedding</code> for direction B. ~90 ms including the embed
9	CLIENT	POST /campaigns/{id}/recommendations/generate — <code>{top_k:10}</code> .
10	ELIGIBILITY	R1–R8 over the 340 discoverable creators in the seeded set. 248 pass, 92 excluded, 7 flagged <code>REVIEW_REQUIRED</code> on ambiguous competitor evidence. 92 rows written to <code>eligibility_exclusions</code> . ~180 ms
11	RETRIEVAL	HNSW search over <code>creator_representations</code> restricted to the eligible ids → 75 candidates with cosine similarities. ~45 ms
12	REPOSITORY	Bulk-load 75 creator bundles: profile, preferences, active rate card, latest snapshot per account, cluster vectors. One query per table, not per creator — the N+1 here is the difference between 200 ms and 9 s. ~120 ms
13	ML	<code>build_pair_features</code> × 75 → DataFrame → <code>xgb-2026-10-17</code> batch predict. ~60 ms batched
14	ML	S_b under the <code>NICHE_CREDIBILITY</code> profile; S_c from each creator's preferences; harmonic combination. Fairness re-rank disabled for this campaign. ~15 ms
15	REPOSITORY	One transaction: 75 <code>match_scores</code> rows with all components and three version strings, plus ~60 <code>match_evidence</code> rows for the top 10. ~140 ms
16	ROUTER	200 OK built from the persisted rows. Top 3: Aarav Fitness 0.90, NutriWithPriya 0.86, MegaLifestyle 0.58. total ≈ 840 ms
17	CLIENT	Discover grid renders. Each card shows three component bars — content fit, audience fit, creator fit — not one percentage.
18	CLIENT	Brand clicks <i>Why this creator?</i> → POST /matches/{c}/{cr}/explain → grounded narrative + the evidence chips with their capture dates. ~1.4 s, cached thereafter
19	SERVICE	POST /campaigns/{id}/invitations → <code>proposals</code> row, <code>initiated_by=BRAND</code> , <code>status=SUBMITTED</code> , <code>match_score_id</code> linked to hop 15.

8.2 Trace B — creator finds the campaign and proposes

The same account switches to creator mode. This is the half of the product that the existing documents leave implicit — and it is the half that makes the reciprocal claim demonstrable.

1	CLIENT	Profile switcher → <code>X-Active-Role: CREATOR</code> . No re-login, no second account. GET /me returns both profiles.
2	CLIENT	GET /creators/{id}/recommendations . Cache is 40 minutes stale → POST /refresh fires in the background while cached results render.
3	SERVICE	Creator blocks first . 86 published campaigns → 74 survive: 9 dropped on <code>excluded_categories</code> , 3 on <code>min_rate_inr</code> . The creator's boundaries are applied before anything else. ~30 ms
4	ELIGIBILITY	Same module, arguments inverted: for each of the 74, does this creator satisfy its requirements? 31 pass. Same <code>rule_version</code> as trace A. ~90 ms
5	RETRIEVAL	Creator vector vs. the 31 cached <code>campaign_query_embedding</code> values — a small in-memory dot product, no ANN needed at this scale. ~8 ms
6	ML	The same <code>build_pair_features</code> , the same model, the same <code>reciprocal()</code> . S_c gate at 0.5 drops 12 more. 19 remain. ~50 ms
7	REPOSITORY	19 <code>match_scores</code> rows with <code>direction=CREATOR_TO_BRAND</code> . Protein Crunch now has two rows — one per direction — with the same S_r and different ranks. ~60 ms
8	ROUTER	200 OK , 8 opportunities. Protein Crunch ranks 1 with S_c 0.87. The card leads with “ <i>INR 18,000 offered · your Reel + 2 Stories = INR 12,000</i> ”. total ≈ 290 ms
9	CLIENT	Creator also sees the brand's invitation from trace A hop 19 in the proposals inbox. Both directions found the same pair — that convergence is the demo moment.
10	AI	POST /ai/proposals/draft → editable text from the campaign requirements, the creator's rate card and the retrieved brand guidelines. Returns “2 Reels + 3 Stories for INR 18,000”. ~1.6 s
11	CLIENT	Creator edits to INR 22,000 and changes deliverables to 1 Reel + 3 Stories. The AI draft is never auto-sent .
12	SERVICE	POST /proposals/{id}/counter → <code>proposal_versions</code> v2, <code>current_version=2</code> , status <code>COUNTER_OFFER</code> . v1 is untouched. ~35 ms
13	CLIENT	Brand mode: the counter appears with both versions side by side and the delta highlighted.
14	SERVICE	POST /proposals/{id}/accept → one transaction: status <code>ACCEPTED</code> , insert <code>collaborations</code> with <code>agreed_amount=22000</code> copied from v2. ~45 ms
15	AI	POST /collaborations/{id}/agreement/draft → six-clause draft grounded in the accepted version plus two guideline chunks; <code>evidence_refs</code> stored. Status <code>DRAFT</code> . ~2.1 s
16	SERVICE	Brand approves → <code>brand_approved_at</code> . Creator approves → <code>creator_approved_at</code> . Only both together move it to <code>APPROVED</code> . The card lands in the <i>In production</i> column.

USE TRACE B AS THE CLOSING DEMO

Most capstone demos in this space show a brand searching and a ranked list appearing. Yours ends with the same pair discovered independently from both sides, a counter-offer that changed the terms, and an agreement neither the model nor the system could execute on its own. That sequence demonstrates reciprocal matching, human-in-the-loop control and grounded generation in about ninety seconds.

UI/UX review of the wireframe package

Eleven frames reviewed, and the findings below were then addressed in the v2 package — thirty frames covering the creator journey, the Wall, Knowledge and Proposals in depth. This part is kept as the reasoning behind each change, because the *why* is what you defend in the viva.

9.1 What is working and should be kept

One account, two modes

Frames 02, 03 and 10 get the hardest product decision right. The mode switcher in the sidebar footer — “Aurum Foods / Brand mode” — is the correct affordance: persistent, low-ceremony, always visible. Build exactly this.

Component bars, not one number

Frame 05's *Content fit 96 / Audience fit 91 / Creator fit 95* is the explainability requirement expressed as UI. Keep the three bars; they map directly onto `match_scores` columns.

Creator fit shown to the brand

A small detail with large meaning: the brand sees how well the campaign suits the *creator*. That is the reciprocal thesis made visible, and no competing product does it. Make it more prominent, not less.

Symmetric collaboration forms

Frame 07 shows both initiation directions side by side with the same mutual-fit banner. This maps one-to-one onto the single `proposals` table with `initiated_by`. Excellent alignment between design and data model.

Agreement with visible approval state

Frame 08's “Brand approval: Confirmed / Creator approval: Awaiting response” is exactly the non-self-executing agreement the architecture requires. It is also the clearest visual proof of human-in-the-loop you have.

Kanban that matches the enum

Frame 09's Proposed / In production / In review columns map onto `collaboration_status` without translation. Keep the values aligned as the schema evolves.

9.2 Changes to make

#	FRAME	ISSUE	CHANGE	PRIORITY
1	04 Wall	The Wall was underspecified, not wrong. “Share an update, opportunity or milestone” left open who posts, who sees it, and whether posting affects matching — which is what made it look like unbounded scope.	Keep it and give it a job: a peer community where brands and creators share ideas, open calls and results, follow each other and reply in threads. Bound it with four post kinds, a chronological feed, no ranking model and no messaging. Add the two rules from §5.12: wall content never reaches the ranking pipeline, and a thread converts into a normal proposal. Then split Home off as a separate activity digest built from rows you already store, so the two surfaces do different work.	RESOLVED
2	05, 07	A single “94% mutual fit” badge contradicts the project’s own principle that recommendations expose components and evidence rather than one percentage. It also invites a precision the data cannot support.	Replace with two numbers and a word: <code>Brand fit 92 · Creator fit 87 · Strong mutual</code> . Keep the component bars. Reserve the single figure for sorting only, never as the headline.	HIGH
3	05	Metrics carry no freshness or provenance. “84K followers 6.8% engage” reads as current truth; it may be a 30-day-old snapshot or a creator-typed figure.	Add a small caption: <code>via YouTube API · 16 Sep</code> or <code>self-reported</code> . Two words per card, and it satisfies both the responsible-AI requirement and the “show missingness and freshness” NFR.	HIGH
4	05	No eligibility narrative. “1,248 eligible creators” is shown but not <i>why</i> the others are not.	Make the stat tile clickable → the exclusions audit view grouped by rule: <i>Geography 190 · Budget 142 · Platform 55 · Creator exclusion 18 · Competitor 7 (review)</i> . This is a two-hour screen that sells the deterministic-filtering design better than any slide.	MEDIUM
5	05	Fit bars encode value by length and a single hue only. At small sizes 89 and 96 are hard to separate, and the encoding is inaccessible.	Keep the numeral beside every bar (already present — make it larger), add a band label (<i>strong / good / borderline</i>), and use <code>tabular-nums</code> so columns align.	MEDIUM
6	06	Step 2 “AI structure” is named in the stepper but never drawn — yet it is the most important screen in the brand journey and the one that enforces “the brand confirms before matching”.	Design it: each extracted field as an editable input, amber border and a <i>please check</i> hint below 0.7 confidence, nulls shown as empty and required, unmapped brief phrases listed as chips, and a primary action reading Confirm requirements — not “Next”.	CRITICAL
7	07	The creator’s form asks for a quote but shows no reference to their own rate card.	Show the computed rate-card total beside the quote field: <i>your card: 1 Reel + 3 Stories = INR 14,500</i> . Negotiation with your own pricing invisible is a strange experience, and the number is already in the response.	MEDIUM
8	08	The AI-generated agreement shows no link from a clause to the evidence it came from, although <code>agreements.evidence_refs</code> stores exactly that.	Make clause 4 (“Content standards”) expandable to the guideline excerpt with its page number. This is the strongest single demonstration of grounded generation in the product.	MEDIUM
9	all	No empty, loading, error or degraded states anywhere. Every frame shows a fully populated happy path.	Each list needs four states. The one that matters most: degraded — “live metrics unavailable, showing data from 16 Sep”. That state is what makes a rate-limited demo look designed rather than broken.	HIGH
10	all	Desktop only, and the sidebar is fixed-width.	You do not need a mobile app, but the demo may be viewed on a laptop at 1280 px or projected. Define one breakpoint where the sidebar collapses to icons and card grids drop to one column.	LOW

9.3 Screens that were missing

Nine of the eleven v1 frames were brand mode. For a product whose thesis is that discovery works in both directions, the creator half needed equal weight. **All twelve are now drawn in the v2 package** — the table stays as the build checklist, with the frame number to work from.

SCREEN	MODE	WHY IT IS NEEDED	FLOW	PRIORITY
Creator profile builder (3 steps)	CREATOR	Nothing enters the system without it	F2	CRITICAL
Rate card editor	CREATOR	Budget eligibility and the whole commercial layer depend on it	F3	CRITICAL
Preferences & exclusions	CREATOR	The entire creator-side utility S_c is computed from this screen	F3	CRITICAL
Opportunity feed — ranked campaigns	CREATOR	Direction B has no surface at all right now	F8	CRITICAL
Campaign detail, creator's view	CREATOR	Requirements, pay vs. own rates, deadline, conflicts	F8	CRITICAL
Confirm extracted requirements	BRAND	Step 2 of a stepper that is already drawn	F6	CRITICAL
“Why this match?” panel	both	FR-08; the explainability claim has no home otherwise	F9	HIGH
Social account connect & sync status	CREATOR	OAuth entry, freshness, disconnect, and the manual fallback	F4	HIGH
Proposal inbox / outbox	both	Frame 07 composes a proposal; nothing shows the ones you have	F10	HIGH
Document upload & policy Q&A	BRAND	FR-11; the RAG demo has no interface	F11	MEDIUM
Exclusions audit	BRAND	Makes deterministic filtering visible and debuggable	F7	MEDIUM
Admin / experiment dashboard	ADMIN	Variant comparison table straight into the report	P10	LOW

9.4 Component inventory

Build these twelve first in `frontend/src/components/`. Every screen above is then an assembly job rather than fresh CSS, which is what lets one person cover twelve screens in three weekends.

COMPONENT	PROPS	USED ON
<ScoreBar>	label, value 0–1, band, tone	every match card, the why-panel
<MutualFitBadge>	brandSide, creatorSide	discover, proposal forms, opportunity feed
<EvidenceChip>	kind, label, provenance, capturedAt	why-panel, agreement clauses
<FreshnessStamp>	capturedAt, provenance	anywhere a provider number appears
<RoleSwitcher>	roles, active, onSwitch	sidebar footer, global
<StatusPill>	status, domain	proposals, collaborations, campaigns, sync
<ConfidenceField>	label, value, confidence, onChange	the confirm-requirements screen
<RateCardTable>	items, editable	creator profile, proposal composer
<EmptyState>	title, body, action	every list
<DegradedBanner>	reason, asOf	whenever an adapter reports degraded
<SourceExcerpt>	filename, page, text, similarity	RAG answers, agreement clauses
<KanbanColumn>	status, items	collaborations

ML and evaluation

The marketplace is the demonstration; this part is the contribution. Six ranking variants over one frozen evaluation set, changing exactly one factor at a time.

10.1 Model ladder

VARIANT	WHAT IT IS	WHAT IT TESTS	BUILT
E1 Popularity	Sort by follower count	Sanity floor. If E1 wins, something is wrong with the labels.	W3
E2 Rule-based	Hard filters + hand-weighted feature score	How far transparent rules get you — the honest baseline	W3
E3 Semantic-only	Cosine similarity alone	Value of meaning without commercial facts	W4
E4 Hybrid, one-sided	Structured + semantic, brand side only	Does combining beat either alone?	W5
E5 Hybrid reciprocal	E4 plus creator-side utility, harmonically combined	The research question. Does two-sided scoring preserve brand relevance while improving mutual suitability?	W5
E6 Reciprocal + rerank	E5 plus MMR fairness re-ranking	What relevance does exposure fairness cost?	W6

Optional ablations if time allows, in priority order: multi-vector creator representation vs. a single averaged embedding; cold-start with history features masked; discovery mode `FRESH_TO_VERTICAL` vs. `CONVENTIONAL`; objective-aware weights vs. fixed universal weights.

10.2 Metrics

QUESTION	METRIC	READ AS
Are the right creators surfaced and ordered?	Precision@5, Recall@5, NDCG@5, MAP	Primary result table
Do both sides benefit?	Mutual NDCG@5 on <code>mutually_relevant</code> ; one-sided mismatch rate; mean joint utility	The reciprocal claim
Does the parser work?	Per-field precision/recall/F1; schema-valid rate; null-handling accuracy	Report core and unseen verticals separately
Are the rules correct?	Rule accuracy, false-exclusion rate , review rate	False exclusion is the dangerous one
Who gets seen?	Exposure share by follower band, catalogue coverage, Gini, intra-list diversity, relevance delta after re-rank	Always report the trade-off, not just the gain
Does it work without history?	NDCG@5 with history features masked; new-creator subset	Cold start
Are explanations trustworthy?	Human clarity/usefulness ratings; factual-consistency rate ; hallucinated-number count	Target zero hallucinated numbers
Is retrieval grounded?	Recall@k, evidence correctness, refusal rate on out-of-scope questions	RAG
Is it practical?	p50/p95 latency per endpoint, cost per AI call, error rate, fallback rate	The “in Practice” part of the course

10.3 Dataset and labels

COMPONENT	TARGET	SOURCE
Creator profiles	150–200 minimum, 300–400 goal — 200 already collected	Your YouTube and Bluesky collectors. Expand toward 340 with a second run; add clearly-labelled <code>SYNTHETIC</code> rate and preference attributes, which is the only part the collectors cannot observe.
Gold consented set	30–50	Real creators giving real rates, exclusions and audience details — each one is worth ten scraped profiles
Campaign briefs	~160: 120 core verticals + 40 unseen	Team-authored from real brand language; the unseen set never informs prompt design
Pair labels	≥ 400 pairs, ≥ 2 raters each	Rubric 0–3 on both sides; mutually relevant when both ≥ 2
RAG corpus	8–12 documents, 30–50 questions	Synthetic brand guidelines with known answers, so correctness is checkable

THREE LEAKAGE RULES — VIOLATING ANY ONE INVALIDATES EVERY NUMBER YOU REPORT

Split by campaign, not by pair. The same campaign appearing in train and test leaks its requirements. **Deduplicate near-identical creators** before splitting — collectors produce near-duplicates and they will straddle the split. **Hold out a final test set and touch it once.** Tune on validation; the test set is opened in week 7 and never again.

10.4 Inter-rater agreement

Five people labelling 400 pairs will disagree, and that disagreement is a result rather than a problem. Have two raters score every pair independently, compute Cohen’s κ or weighted κ per side, and adjudicate disagreements greater than one point in a recorded session. Report κ in the paper — a reciprocal ranking evaluated against labels of unknown reliability is not evidence, and showing you measured it is exactly the rigour the course asks for.

Security, observability and delivery

Short, non-negotiable, and mostly enforced by a test rather than by discipline.

11.1 Security controls

RISK	CONTROL	ENFORCED BY
OAuth token leakage	Encrypted at rest with <code>OAUTH_TOKEN_ENCRYPTION_KEY</code> ; never serialised into any response; redacted in logs	A test asserting no response body ever contains <code>token</code>
Secrets in git	<code>.env</code> ignored, <code>.env.example</code> committed with empty values	Already correct – verify <code>instagram_data.json</code> holds no token
Cross-tenant reads	Every repository call filters by owner; ownership asserted in <code>deps</code> , not in the route body	A test that brand B cannot read brand A's documents or matches
PII into prompts	Only task-required fields reach a model; ids and aggregates preferred over names	Typed prompt payloads – if it is not on the <code>MatchEvidence</code> model, it cannot be sent
Prompt injection via uploads	Retrieved chunks delimited and labelled as reference data; never routed into a tool-calling context	A fixture PDF containing “ignore previous instructions” in the test suite
Hallucinated metrics	Numbers in generated text validated against the evidence payload before returning	The check in F9 step 3
Sensitive-trait inference	No religion, health, politics or similar inferred from content; category vocabulary is fixed and reviewed	Vocabulary constant in code review
Stale data shown as current	<code>captured_at</code> rendered wherever a provider number appears	The <code><FreshnessStamp></code> component
Autonomous commercial action	No state transition without an authenticated human POST	The state machine in F10

11.2 Observability

- A `correlation_id` per request, generated in middleware, returned in the response and every error, and written to `ai_execution_logs`. One id ties a user report to a model call.
- Every AI call logs provider, model, prompt version, latency, token estimate, status and error category. This table is your practice-metrics section.
- Every recommendation run logs feature, model and representation versions plus candidate counts at each stage. When ranking quality changes, you will know which stage moved.
- Social syncs log the request class, outcome and any quota error – quota exhaustion should never present as a mysterious empty profile.
- Prometheus and Grafana are explicitly out of scope. Structured logs plus one summary notebook meet the requirement.

11.3 Environment variables to add

.ENV.EXAMPLE – APPEND

```
JWT_ACCESS_TTL_MINUTES=30
JWT_REFRESH_TTL_DAYS=7
OAUTH_TOKEN_ENCRYPTION_KEY=          # 32-byte urlsafe base64 – generate per environment
DOCUMENT_STORAGE_PATH=./storage/documents
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
EMBEDDING_DIM=384
LLM_MODEL=
LLM_TIMEOUT_SECONDS=30
LLM_MAX_RETRIES=1
RECOMMENDATION_TOP_N=75
RECOMMENDATION_TOP_K=10
CREATOR_SIDE_GATE=0.5
MIN_TRANSFERABLE_FIT=0.62
BSKY_IDENTIFIER=
BSKY_APP_PASSWORD=
```

11.4 Git and CI

- Protect `main`. Branches named `feature/<scope>`, `fix/<issue>`, `spike/<experiment>`. Issue → branch → tests → PR → one teammate review → merge. This is already in `CONTRIBUTING.md`; follow it.
- Every PR states what changed, which issue it closes, how it was tested, and a screenshot or a sample API response. No screenshot on a frontend PR means no merge.
- CI on every PR: `npm ci → lint → test → build` and `pip install → ruff → pytest`. Add `alembic upgrade head` against a throwaway Postgres service from weekend 2 – a migration that only works on one laptop is the classic week-6 disaster.
- Add ADRs 011 (rejected technologies), 012 (dual-role identity), 013 (Bluesky as a third adapter), 014 (wall reframed as activity digest). Your `docs/decisions/` convention already exists; keep using it.

Ownership and definition of done

Five workstreams with clean interfaces, so five people can work in parallel without waiting on each other. The weekend-by-weekend schedule lives in the companion execution plan.

12.1 Workstreams and interfaces

WORKSTREAM	OWNS	PUBLISHES TO OTHERS	CONSUMES
A • Platform & Backend	Auth, deps, schema, migrations, repositories, campaign and profile services, error shape, CI	The OpenAPI schema and the migration chain — everyone else builds against these	Typed score objects from C; normalised contracts from E
B • Frontend & Product	Every screen, the component kit, auth context, role switching, UX acceptance criteria	Screenshots on every PR; UX bug reports	OpenAPI from A; MSW mocks until endpoints land
C • ML & Evaluation	Feature builder, model ladder, reciprocal scoring, re-ranking, metrics, experiment log	<code>build_pair_features</code> , <code>predict()</code> , <code>reciprocal()</code> — pure functions with version strings	Bundles from A; labels from D
D • GenAI, RAG & Data	LLM adapter, brief parser, explanations, RAG ingest and retrieval, embeddings, LangGraph, label rubric	Typed extraction and explanation objects; the label dataset	Evidence objects from A; encoder shared with C
E • Integrations & QA	The three adapters, OAuth, sync jobs, seed data, integration tests, demo script, deployment, report	The normalised social contract; the seeded demo database	Tables from A

PROVISIONAL MAPPING FROM THE REPOSITORY'S OWN HISTORY

Harshit owns the repo, the architecture documents and the transformed-creator contract → **A**. Manjusha shipped the YouTube collector branch → **E**. The Bluesky integration branch author → **C** or **E**. Mohit wrote the creator data strategy and the sub-problem analysis → **C**. The remaining member takes **B** or **D**. Settle this in weekend 1 and record it in the README — the individual viva rewards a clear, evidenced contribution log, so start it on day one rather than reconstructing it in December.

12.2 Definition of done

KIND OF WORK	DONE MEANS
Backend feature	Migration if the schema changed · service/repository separation held · Pydantic in and out · unit + integration test · OpenAPI regenerated · error cases return the common shape
Frontend feature	Works against the real API · loading, empty, error and degraded states · role authorisation respected · keyboard-reachable · screenshot in the PR
ML experiment	Dataset, feature and model versions recorded · held-out evaluation · metrics plus error analysis · a script that reproduces the number from scratch
AI / RAG feature	Typed input and output · prompt version pinned · grounded test set passes · defined failure behaviour · no uncontrolled action
Integration	OAuth round-trip tested · secrets safe · normalised mapping tested against a fixture · cached fallback works · expiry and disconnect documented
Release	<code>docker compose up --build</code> from a clean clone · CI green · seed/demo mode works offline · known limitations written down

12.3 The first twelve issues to open

#	ISSUE	STREAM	BLOCKS
1	Migration 0002: dual-role identity, backfill, drop <code>users.role</code>	A	everything
2	JWT issue/verify + <code>deps.py</code> + register/login/refresh/me	A	everything
3	<code>core/errors.py</code> + the common error shape handler	A	all endpoints
4	Auth context, token store, role switcher, protected routes	B	all screens
5	MSW mock layer from the OpenAPI schema	B	unblocks B from A
6	Migration 0003 + creator profile CRUD (F2)	A	F3, F5, F7
7	Creator profile wizard, 3 steps	B	seeding real data
8	Refactor the three collectors behind <code>SocialAdapter</code>	E	F4
9	Shrink <code>data/raw/</code> below 40 MB, then <code>scripts/seed_data.py</code> : load the 200 collected creators + 20 campaigns	E	C cannot start without it
10	Label rubric v1 + the first 50 labelled pairs	D	C's whole ladder
11	<code>build_pair_features</code> + unit tests against fixtures	C	E1–E6
12	Eligibility service R1–R8 + exclusion rows + tests	A/C	F7, F8

IF THE SEVEN WEEKENDS COMPRESS

Protect this spine and cut everything else: dual-role auth → creator profile with rate card and preferences → campaign with confirmed requirements → deterministic eligibility → semantic retrieval → E2 vs. E4 vs. E5 with real metrics → both discovery directions → the proposal state machine end to end. That is a complete, defensible capstone. LangGraph, RAG, fairness re-ranking, agreements and the activity digest are all genuinely optional, and a project that ships the spine cleanly scores better than one that half-builds all twelve flows.

BrandBridge Build Manual v1.0 · compiled 20 September 2026 from the Consolidated Project Plan V3, the Technical Architecture & Implementation Guide, the Creator Profile & Data Strategy, the Business & Academic Structure, the UI wireframe package, and a direct read of the repository at commit `caff3d8` .

Revalidate all external API details — YouTube quotas, Meta permissions, LLM provider pricing — in the provider dashboards immediately before final implementation. The internal normalised contracts in Part 5 exist so that provider changes never reach the ML or UI layers.